



**UNIVERSITY OF ENERGY AND NATURAL  
RESOURCES, SUNYANI**

**DEPARTMENT OF INFORMATION TECHNOLOGY AND  
DECISION SCIENCES**

**DEVELOPMENT OF AN AI-POWERED STUDY ASSISTANT FOR  
UNDERGRADUATE STUDENTS USING ADAPTIVE ASSESSMENT-  
BASED LEARNING**

**Group 23D**

Chaluatera Calvin Akiweley - UEB3256322

Bawuah Andy Michael - UEB3261022

Joyce Nsiah Atogmbune - UEB3257422

Asiak Angela Anaamlie - UEB3254022

Njourkoneh Prosper - UEB3265322

**Supervisor: Mr. Sarfo Manu**

Academic Year: 2025/2026

**ABSTRACT**

AI is everywhere in university life these days. You can't walk through campus without hearing someone talk about ChatGPT or Grammarly. The Digital Education Council (2024) found that 86% of students across 16 different countries are using AI for schoolwork, and over half use it at least once a week. In the US, Contractor and Reyes (2025) found that more than 80% of students at one college were already using AI only two years after ChatGPT came out.

In Ghana, Adarkwah et al. (2023) showed that university staff and students are quickly warming up to AI like ChatGPT and MetaAI. But here is the problem: Obenza et al. (2024) found that most students just ask AI for answers instead of using it to actually test themselves.

So our project tries to fix that. We built a web-based AI study assistant where students upload their own materials lecture notes, slides, even photos of handwritten notes. The system uses OCR to read handwriting, then an AI model creates quiz questions, exam-style theory questions, and flashcards from that specific content. There's also a chatbot that only answers based on what the student uploaded. After a quiz, the student gets a feedback report showing exactly where they need to improve.

The system uses HTML, CSS, JavaScript, PHP, MySQL, Tesseract OCR, and the NVIDIA LLM API. We looked at seven studies from 2020 to 2025, and none of them combined all these features into one free platform for African university students.

Keywords: AI study assistant, adaptive assessment, question generation, OCR, flashcards, LLM, higher education, Ghana.

## **DEDICATION**

We dedicate this project first and foremost to Almighty God for His grace, wisdom, strength, and guidance throughout our academic journey and during the development of this project.

We also dedicate this work to our beloved parents and guardians whose unwavering support, sacrifices, encouragement, and prayers have been a constant source of motivation. Their commitment to our education has made this achievement possible.

Finally, we dedicate this project to all students who seek innovative ways to enhance their learning experience and academic success through technology.

## **ACKNOWLEDGEMENT**

We express our sincere gratitude to Almighty God for granting us the knowledge, strength, good health, and perseverance required to successfully complete this project.

We would like to extend our heartfelt appreciation to our project supervisor, Mr. Sarfo Manu, for his invaluable guidance, constructive criticisms, encouragement, and continuous support throughout the development of this project. His expertise and dedication greatly contributed to the success of this work.

Our profound gratitude also goes to the Head of the Department (HOD) Professor Peter Appiahene and the entire staff of the Department of Information Technology and Decision Sciences, University of Energy and Natural Resources, Sunyani, for providing us with the academic foundation, resources, and support necessary for completing this project.

We are deeply grateful to our parents and guardians for their sacrifices, financial assistance, patience, prayers, and unwavering encouragement throughout our educational journey. Their support has been instrumental in helping us achieve our academic goals.

Finally, we appreciate our colleagues, friends, and everyone who contributed directly or indirectly to the successful completion of this project. May God richly bless you all.

## **TABLE OF CONTENTS**

### **CHAPTER ONE: INTRODUCTION**

|                                     |   |
|-------------------------------------|---|
| 1.1 Background of the Study .....   | 1 |
| 1.2 Problem Statement .....         | 2 |
| 1.3 Aim of the Study .....          | 3 |
| 1.4 Objectives of the Study .....   | 3 |
| 1.5 Research Questions .....        | 4 |
| 1.6 Significance of the Study ..... | 4 |
| 1.7 Scope of the Study .....        | 5 |
| 1.8 Organization of the Study ..... | 6 |

### **CHAPTER TWO: LITERATURE REVIEW**

|                                                       |    |
|-------------------------------------------------------|----|
| 2.1 Introduction .....                                | 7  |
| 2.2 Artificial Intelligence in Education (AIED) ..... | 7  |
| 2.3 Intelligent Tutoring Systems (ITS) .....          | 8  |
| 2.4 Adaptive Learning Systems .....                   | 9  |
| 2.5 Machine Learning in Educational Systems .....     | 10 |
| 2.6 Adaptive Assessment-Based Learning .....          | 11 |
| 2.7 Review of Existing Systems .....                  | 12 |
| 2.8 Theoretical Framework .....                       | 13 |
| 2.9 Conceptual Framework .....                        | 14 |
| 2.10 Research Gap .....                               | 15 |

## **CHAPTER THREE: METHODOLOGY**

|                                          |    |
|------------------------------------------|----|
| 3.1 Research Design .....                | 16 |
| 3.2 System Development Methodology ..... | 16 |
| 3.3 Requirements Analysis .....          | 17 |
| 3.4 System Architecture .....            | 19 |
| 3.5 Data Collection Methods .....        | 20 |
| 3.6 Database Design .....                | 21 |
| 3.7 AI Techniques to be Used .....       | 22 |
| 3.8 Tools and Technologies .....         | 23 |
| 3.9 System Testing .....                 | 24 |

## **CHAPTER FOUR: SYSTEM DEVELOPMENT AND IMPLEMENTATION**

|                                    |    |
|------------------------------------|----|
| 4.1 System Analysis .....          | 25 |
| 4.2 System Design .....            | 26 |
| 4.3 User Interface Design .....    | 27 |
| 4.4 System Implementation .....    | 28 |
| 4.5 Database Implementation .....  | 30 |
| 4.6 System Security Measures ..... | 31 |

## **CHAPTER FIVE: TESTING, EVALUATION, CONCLUSION AND RECOMMENDATIONS**

|                                  |    |
|----------------------------------|----|
| 5.1 System Testing Results ..... | 32 |
| 5.2 System Evaluation .....      | 33 |
| 5.3 Findings .....               | 34 |
| 5.4 Conclusion .....             | 35 |
| 5.5 Recommendations .....        | 36 |

## **REFERENCES**

|                  |    |
|------------------|----|
| References ..... | 37 |
|------------------|----|

## APPENDICES

|                                       |    |
|---------------------------------------|----|
| Appendix A: Questionnaires .....      | 38 |
| Appendix B: System Screenshots .....  | 39 |
| Appendix C: Source Code Samples ..... | 40 |
| Appendix D: User Manual .....         | 41 |
| Appendix E: Test Results .....        | 42 |

## Expected Deliverables

Requirements Specification Document  
System Design Document  
AI-Powered Study Assistant Web Application  
Database System  
User Manual  
Test Report  
Final Project Report  
Presentation Slides  
Source Code Repository

## CHAPTER ONE: INTRODUCTION

### 1.1 Background of the Study

The rapid advancement of Artificial Intelligence (AI) has significantly transformed various sectors, including education. AI technologies are increasingly being integrated into learning environments to provide personalized, efficient, and engaging educational experiences. Traditional learning systems often adopt a one-size-fits-all approach, which may not adequately address the diverse learning needs, abilities, and paces of undergraduate students. As a result, many students struggle to achieve optimal academic performance due to a lack of individualized support and continuous assessment.

In higher education institutions, students are expected to manage large volumes of academic content while developing critical thinking and problem-solving skills. However, conventional study methods and learning management systems frequently provide limited personalization and feedback, making it difficult for students to identify their strengths, weaknesses, and learning gaps. This challenge has created a growing demand for intelligent learning tools that can adapt to individual student needs and improve learning outcomes.

Adaptive assessment-based learning has emerged as an effective educational approach that continuously evaluates a learner's performance and adjusts learning content, difficulty levels, and recommendations accordingly. By analyzing assessment results, adaptive systems can identify areas where students require additional support

and provide tailored learning resources to enhance understanding and retention. Such systems promote self-paced learning, increase student engagement, and improve academic achievement.

The integration of AI into adaptive learning environments further enhances the effectiveness of these systems. AI-powered study assistants can utilize machine learning algorithms, natural language processing, and data analytics to offer personalized study recommendations, generate practice questions, provide instant feedback, and monitor learning progress. These intelligent features enable students to receive real-time academic support while helping educators better understand student performance patterns.

Despite the growing adoption of AI in education, many existing study applications focus primarily on content delivery and lack robust adaptive assessment mechanisms that dynamically personalize learning experiences based on student performance. Furthermore, some systems provide limited interaction and fail to offer comprehensive feedback tailored to individual learning needs. These limitations highlight the need for more intelligent and interactive educational tools that combine adaptive assessment with AI-driven personalization.

Therefore, this study focuses on the development of an Interactive AI-Powered Study Assistant for Undergraduate Students Using Adaptive Assessment-Based Learning. The proposed system aims to provide personalized learning experiences by continuously assessing student performance, identifying knowledge gaps, and recommending appropriate learning materials and assessments. By leveraging AI technologies, the system seeks to improve student engagement, learning effectiveness, and overall academic success in higher education environments.

## **1.2 Problem Statement**

Undergraduate students face numerous academic challenges, including difficulty understanding complex course materials, ineffective study habits, inadequate access to personalized learning support, and limited opportunities for continuous self-assessment. Traditional learning methods and many existing e-learning platforms often provide the same learning content and assessment structure to all students regardless of their individual learning abilities, preferences, and academic performance. This generalized approach may lead to poor knowledge retention, reduced engagement, and unsatisfactory academic outcomes.

Although various digital learning platforms and study applications are available, many lack intelligent mechanisms for adapting learning content and assessments to the specific needs of individual learners. Students frequently receive delayed feedback on their performance, making it difficult to identify and address learning gaps promptly. Additionally, existing systems often provide limited interaction and fail to offer personalized recommendations that can guide students toward effective learning

strategies and resources.

The increasing complexity of university curricula and the growing demand for flexible learning solutions further emphasize the need for intelligent educational support systems. Without adaptive learning assistance, students may spend excessive time studying topics they already understand while neglecting areas where they require improvement. This can result in inefficient learning processes and lower academic performance.

Therefore, there is a need for an interactive AI-powered study assistant that utilizes adaptive assessment-based learning to continuously evaluate student performance, identify knowledge gaps, and provide personalized learning recommendations. Such a system can enhance student engagement, improve learning efficiency, and support better academic achievement by delivering customized educational experiences tailored to individual learner needs.

### **1.3 Aim of the Study**

The aim of this study is to design and develop an Interactive AI-Powered Study Assistant for Undergraduate Students using Adaptive Assessment-Based Learning to provide personalized learning support, continuously assess student performance, identify knowledge gaps, and recommend appropriate learning resources that enhance student engagement, learning effectiveness, and academic achievement.

### **1.4 Objectives of the Study**

#### **Main Objective**

The main objective of this study is to design and implement an Interactive AI-Powered Study Assistant for Undergraduate Students using Adaptive Assessment-Based Learning that delivers personalized learning experiences through continuous assessment, intelligent feedback, and adaptive study recommendations.

#### **Specific Objectives**

To analyze the learning challenges faced by undergraduate students and identify requirements for an AI-powered study assistant.

To design a user-friendly system that enables students to interact with an intelligent study assistant for academic support.

To develop an adaptive assessment module that evaluates student performance and identifies knowledge gaps.

To implement AI techniques for generating personalized study recommendations and learning pathways based on individual performance.

To provide real-time feedback and progress tracking to help students monitor and

improve their academic performance.

To test and evaluate the effectiveness, usability, and performance of the developed system in supporting undergraduate learning.

## **1.5 Research Questions**

The study seeks to answer the following research questions:

1. What learning challenges do undergraduate students face that can be addressed through an AI-powered study assistant?

2. How can an interactive AI-powered study assistant be designed and developed to support undergraduate learning effectively?

How can adaptive assessment techniques be used to adjust learning content and question difficulty based on student performance?

How can the system identify students' strengths, weaknesses, and knowledge gaps through continuous assessment?

To what extent can personalized learning recommendations improve students' learning experiences and academic performance?

How effective and usable is the developed AI-powered study assistant in enhancing student engagement and learning outcomes?

## **1.6 Significance of the Study**

This study is significant because it contributes to the advancement of educational technology by integrating Artificial Intelligence (AI) and adaptive assessment-based learning to enhance the learning experience of undergraduate students. The findings and developed system are expected to benefit various stakeholders in the educational sector.

### **1. Undergraduate Students**

The study will provide students with a personalized learning environment that adapts to their individual learning needs and abilities. The AI-powered study assistant will help students identify their strengths and weaknesses, receive instant feedback, and access tailored learning resources, thereby improving academic performance and study efficiency.

### **2. Lecturers and Educators**

The system will assist lecturers in monitoring student progress and identifying areas where students experience learning difficulties. Insights generated from student assessments can help educators make informed decisions regarding teaching strategies and academic support.

### **3. Educational Institutions**

Universities and colleges can utilize the developed system to enhance the quality of teaching and learning. The adoption of AI-driven educational tools can contribute to improved student engagement, retention rates, and overall academic success.

#### **4. Researchers and Future Developers**

This study will serve as a valuable reference for researchers and software developers interested in Artificial Intelligence, adaptive learning systems, and educational technologies. It provides a foundation for future studies and innovations in intelligent learning support systems.

#### **5. Advancement of AI in Education**

The study contributes to the growing body of knowledge on the application of AI in education. It demonstrates how adaptive assessment and intelligent recommendation techniques can be used to create more effective and personalized learning experiences.

### **1.7 Scope of the Study**

This study focuses on the design and implementation of an Interactive AI-Powered Study Assistant for Undergraduate Students using Adaptive Assessment-Based Learning. The system is intended to support undergraduate students by providing personalized learning assistance, adaptive assessments, real-time feedback, and study recommendations based on individual performance.

The study will cover the development of key functionalities such as user registration and authentication, course and topic selection, adaptive quizzes and assessments, performance tracking, AI-driven study recommendations, and progress monitoring. The system will analyze students' assessment results to identify learning gaps and adjust the difficulty level of subsequent questions accordingly.

The implementation will be limited to undergraduate-level academic content and selected courses used for system testing and evaluation. The study will primarily focus on the application of AI techniques for personalization and adaptive learning rather than the development of advanced AI models from scratch.

Furthermore, the evaluation of the system will be conducted based on criteria such as usability, effectiveness, user satisfaction, and its ability to support personalized learning. The study will not cover institution-wide deployment, integration with existing university learning management systems, or support for postgraduate and non-academic learning environments.

Therefore, the scope of this study is confined to the development and assessment of an

AI-powered adaptive study assistant designed specifically to enhance the learning experience and academic performance of undergraduate students.

## **CHAPTER 2: LITERATURE REVIEW**

### **2.1 Introduction**

Artificial Intelligence (AI) has become one of the most transformative technologies in modern education, offering innovative solutions to address the diverse learning needs of students. The increasing demand for personalized and flexible learning experiences has led to the development of intelligent educational systems that can adapt to individual learners' abilities, preferences, and academic performance. Among these innovations, AI-powered study assistants and adaptive learning systems have gained significant attention due to their potential to enhance student engagement, knowledge retention, and overall academic achievement.

Traditional educational approaches often rely on standardized teaching methods and assessments that may not adequately accommodate differences in students' learning styles and levels of understanding. Consequently, some students may struggle to keep pace with course requirements, while others may not be sufficiently challenged. Adaptive assessment-based learning addresses this challenge by continuously evaluating learner performance and adjusting instructional content and assessment difficulty to meet individual needs.

AI-powered study assistants further enhance adaptive learning environments by providing intelligent support through personalized recommendations, automated feedback, performance monitoring, and interactive learning experiences. These systems utilize technologies such as machine learning, natural language processing, and data analytics to understand learner behavior and deliver customized educational support. As a result, students can receive targeted assistance that helps them overcome learning difficulties and improve their academic outcomes.

This chapter reviews existing literature related to Artificial Intelligence in education, adaptive learning systems, intelligent tutoring systems, adaptive assessment techniques, and AI-powered study assistants. It also examines relevant theoretical and empirical studies, identifies strengths and limitations of existing solutions, and highlights research gaps that justify the development of an Interactive AI-Powered Study Assistant for Undergraduate Students Using Adaptive Assessment-Based Learning. The chapter provides the conceptual foundation upon which the proposed system is designed and implemented.

## **2.2 Artificial Intelligence in Education (AIED)**

### **2.2.1**

Artificial Intelligence in Education is the interdisciplinary research “at the frontiers of computer science, education and psychology. It promotes rigorous research and development of interactive and adaptive learning environments for learners of all ages, across all domains” (International AIED Society, 2010).

Recent research defines AIED as systems capable of analyzing learner data, identifying learning patterns, and adapting instructional content in real time to enhance student performance and engagement (Boulhrir et al., 2026). Unlike traditional systems, AIED focuses on continuous personalization and intelligent decision-making throughout the learning process.

### **2.2.2 Applications of AI in Education**

AI is widely applied across different educational domains to enhance both teaching and learning experiences.

#### **1. Intelligent Tutoring Systems**

Intelligent tutoring systems provide automated, personalized instruction similar to human tutors. These systems adapt content based on learner performance and provide real-time feedback (Hoogers et al., 2025).

#### **2. Automated Assessment Systems**

AI-based assessment tools can automatically grade quizzes, assignments, and exams while providing instant feedback to students, improving learning speed and accuracy (Wang et al., 2024).

#### **3. Learning Analytics**

Learning analytics uses AI to analyze student behavior and academic performance to support decision-making and early intervention strategies (Siemens, 2023).

#### **4. AI Chatbots and Virtual Assistants**

Schmitz, A. (2025) defines AI-powered chatbots as conversational agents that deliver

round-the-clock academic support, enabling students to access explanations and problem-solving assistance beyond traditional classroom hours.

## **5. Recommendation Systems**

These systems suggest personalized learning materials such as videos, notes, and quizzes based on student behavior and performance history (Gligorea et al., 2023).

### **2.2.3 Benefits of AIED**

AI in education offers numerous benefits that improve both learning and teaching processes:

- Personalized learning experiences tailored to student needs (Holmes et al., 2020)
- Improved student engagement through interactive systems
- Immediate feedback that enhances understanding (Wang et al., 2024)
- Reduced workload for educators through automation
- Data-driven decision-making in academic institutions

These benefits contribute to improved academic performance and learner satisfaction.

### **2.2.4 Limitations of AIED**

Despite its advantages, AIED also faces several challenges:

- High development and implementation cost
- Data privacy and ethical concerns (Porayska-Pomsta et al., 2024)
- Algorithmic bias affecting fairness in learning recommendations
- Limited human emotional understanding in AI systems
- Dependence on high-quality data for accurate predictions

## **2.3 Intelligent Tutoring Systems (ITS)**

### **2.3.1 Overview of ITS**

**Krishna Kumari, R., & Snehalatha, M. (2024)** define Intelligent Tutoring Systems as computer-based educational platforms that utilize artificial intelligence to provide personalized and adaptive learning experiences, assessing individual student progress, tailoring instructional content to meet specific needs, and offering real-time feedback to

create an interactive and customized learning environment that mimics the role of a human tutor.

ITS are considered one of the most effective applications of AI in education due to their ability to support individualized learning at scale (Yuensook et al., 2024).

### **2.3.2 Architecture of ITS**

The architecture of ITS consists of four main components:

#### **1. Domain Model**

Represents subject knowledge, including concepts, rules, and learning materials. It ensures content accuracy and structure (Gligorea et al., 2023).

#### **2. Student Model**

Tracks learner knowledge, performance, and behavior. It is continuously updated based on student interactions (Boulhrir et al., 2026).

#### **3. Pedagogical Model**

Determines how content is delivered, including feedback, hints, and teaching strategies (Hoogers et al., 2025).

#### **4. User Interface**

Enables interaction between the learner and the system, presenting questions, feedback, and learning materials.

### **2.3.3 Student Modeling**

Student modeling is the process of representing learner knowledge, skills, and progress. It enables personalization by identifying strengths and weaknesses in real time (Wang et al., 2024).

Modern ITS use machine learning to continuously update student models, improving accuracy in predicting learning needs and performance outcomes.

### **2.3.4 Domain Modeling**

Domain modeling organizes educational content into structured knowledge units. It defines relationships between topics and ensures logical progression in learning materials (Hoogers et al., 2025).

A strong domain model improves the effectiveness of adaptive systems by ensuring accurate content delivery.

### **2.3.5 Pedagogical Modeling**

Sulisworo, D., & Arslan, A. (2025) define pedagogical modeling as the component of intelligent tutoring systems that determines how content is delivered, including the selection of appropriate actions such as feedback and hints, based on the learner's interactions, response and correctness, and learner profile.

It ensures that learners receive appropriate guidance based on their current level of understanding.

## **2.4 Adaptive Learning Systems**

### **2.4.1 Adaptive Learning Concepts**

Adaptive learning systems use AI to adjust learning content based on student performance and behavior. These systems continuously analyze learner data to provide personalized instruction (Gligorea et al., 2023).

They ensure that learners progress at their own pace while receiving appropriate support when needed (Wang et al., 2024).

### **2.4.2 Personalized Learning Pathways**

Personalized learning pathways refer to customized learning routes designed based on student performance, preferences, and goals. These pathways allow learners to follow individualized study plans rather than fixed curricula (Hoogers et al., 2025).

AI systems dynamically adjust learning sequences to optimize understanding and retention.

### **2.4.3 Adaptive Assessment Techniques**

Adaptive assessment techniques adjust question difficulty based on learner responses. Computerized adaptive testing (CAT) is commonly used to measure student ability more accurately than traditional assessments (Gligorea et al., 2023).

These systems reduce test length while improving assessment accuracy and fairness (Wang et al., 2024).

## **2.5 Machine Learning in Educational Systems**

### **2.5.1 Student Performance Prediction**

Machine learning models are used to predict student academic performance based on historical and behavioral data. These models help identify students at risk of poor performance and support early intervention (Wang et al., 2024).

Common algorithms include neural networks, decision trees, and support vector machines.

### **2.5.2 Recommendation Systems**

Recommendation systems suggest learning materials based on student behavior and preferences. Content-based filtering recommends similar resources, while collaborative filtering uses patterns from other learners (Gligorea et al., 2023).

Hybrid systems improve accuracy by combining both approaches (Hoogers et al., 2025).

### **2.5.3 Learning Analytics**

Learning analytics involves analyzing educational data to improve learning outcomes. AI techniques are used to identify patterns, predict performance, and support decision-making (Siemens, 2023).

It helps educators monitor student progress and implement timely interventions (Wang et al., 2024).

## **2.6 Adaptive Assessment-Based Learning**

Adaptive assessment-based learning refers to an intelligent educational approach where assessment is continuously used to guide, modify, and personalize the learning process. Unlike traditional assessment methods that evaluate students only at the end of a learning period, adaptive assessment integrates evaluation into the learning cycle to improve understanding, provide feedback, and adjust instructional content in real time (Gligorea et al., 2023; Wang et al., 2024).

This approach is a key foundation of AI-powered learning systems because it enables continuous monitoring of learner progress and supports personalized learning pathways.

### **2.6.1 Diagnostic Assessment**

Diagnostic assessment is the initial stage of adaptive learning used to determine a learner's existing knowledge, strengths, weaknesses, and readiness before instruction begins. It helps the system establish a baseline understanding of the student's academic level.

In AI-based educational systems, diagnostic assessments are used to categorize learners into appropriate difficulty levels and recommend suitable learning materials (Hoogers et al., 2025).

#### **Importance of Diagnostic Assessment**

- Identifies prior knowledge gaps
- Determines starting learning level
- Supports personalized learning pathway creation
- Helps prevent unnecessary repetition of known content

Recent studies show that diagnostic assessment improves the accuracy of adaptive systems by ensuring that learning content is aligned with learner ability from the beginning (Wang et al., 2024).

### **2.6.2 Formative Assessment**

Formative assessment is a continuous evaluation process conducted during learning to monitor progress and provide feedback. It is one of the most important components of adaptive learning systems because it supports real-time improvement.

In AI-powered systems, formative assessment includes quizzes, practice exercises, interactive questions, and automated feedback mechanisms that help students learn while being assessed (Gligorea et al., 2023).

### **Key Features of Formative Assessment**

- Continuous feedback during learning
- Identification of misconceptions
- Immediate correction of errors
- Adjustment of learning content based on performance

Formative assessment enhances engagement and improves retention by allowing learners to learn from mistakes immediately (Yuensook et al., 2024).

## **2.6.3 Summative Assessment**

Summative assessment evaluates student learning at the end of an instructional unit, course, or module. It is used to measure overall achievement and determine whether learning objectives have been met.

In adaptive learning systems, summative assessments are used to validate the effectiveness of personalized learning pathways and measure final performance outcomes (Hoogers et al., 2025).

### **Importance of Summative Assessment**

- Measures overall learning achievement
- Provides final performance evaluation
- Supports grading and certification
- Validates learning outcomes of adaptive systems

Although summative assessments are less flexible than formative assessments, they remain essential for academic evaluation and system effectiveness measurement (Wang et al., 2024).

## **2.6.4 Dynamic Difficulty Adjustment**

Dynamic Difficulty Adjustment (DDA) is a core feature of adaptive learning systems where the difficulty level of questions or learning tasks changes automatically based on learner performance.

If a student answers correctly, the system increases difficulty; if the student struggles, the system reduces difficulty or provides additional support (Gligorea et al., 2023).

### **How Dynamic Difficulty Adjustment Works**

- Begins with medium-level questions
- Correct answers increase difficulty level
- Incorrect answers reduce difficulty level
- System continuously recalibrates learner ability

### **Benefits of Dynamic Difficulty Adjustment**

- Maintains optimal learning challenge
- Prevents frustration or boredom
- Improves learner engagement
- Ensures personalized learning progression

## **2.7 Review of Existing Systems**

Several existing educational platforms and learning management systems have been developed to support online learning, self-paced study, and digital assessment. However, these systems vary in their level of personalization, use of Artificial Intelligence, assessment strategies, and feedback mechanisms. This section reviews some widely used systems, including Duolingo, Khan Academy, Moodle, and Coursera.

### **2.7.1 Duolingo**

Duolingo is a popular language-learning platform that uses gamification and adaptive learning techniques to support user engagement. The system applies AI-driven algorithms to adjust lesson difficulty based on learner performance and consistency.

#### **Strengths**

- Provides adaptive learning paths based on user progress
- Uses gamification to increase engagement
- Offers immediate feedback on answers
- Mobile-friendly and accessible

#### **Limitations**

- Limited to language learning only
- AI personalization is relatively shallow compared to advanced tutoring systems
- Does not support deep academic subject tutoring

### **2.7.2 Khan Academy**

Khan Academy is a free online learning platform that provides instructional videos, practice exercises, and progress tracking across multiple academic subjects.

#### **Strengths**

- Offers structured learning content across many subjects

Provides practice exercises with instant feedback  
Includes basic mastery-based learning approach  
Free and accessible globally

**Limitations**

Limited adaptive intelligence compared to modern AI systems  
Personalization is rule-based rather than fully AI-driven  
Does not dynamically adjust question difficulty in real time at a deep level

### **2.7.3 Moodle**

Moodle is an open-source Learning Management System (LMS) widely used by educational institutions for course delivery and management.

**Strengths**

Highly customizable and widely adopted in universities  
Supports quizzes, assignments, and grading  
Provides structured course management tools  
Extensible through plugins

**Limitations**

Limited built-in AI capabilities  
Lacks adaptive learning and intelligent tutoring features by default  
Requires external plugins for advanced personalization

### **2.7.4 Coursera**

Coursera is an online learning platform offering courses, certifications, and degree programs from universities and organizations worldwide.

**Strengths**

High-quality academic content from institutions  
Offers structured courses and certifications  
Includes quizzes and peer-reviewed assessments  
Some personalization through recommendation systems

**Limitations**

Limited real-time adaptation during learning  
Feedback is often delayed or static  
AI-based personalization is minimal compared to intelligent tutoring systems

### **2.7.5 Comparative Analysis of Existing Systems**

The comparison of Duolingo, Khan Academy, Moodle, and Coursera is based on key criteria relevant to this study: personalization, AI capabilities, assessment methods, and feedback mechanisms.

| Criteria           | Duolingo         | Khan Academy       | Moodle               | Coursera        |
|--------------------|------------------|--------------------|----------------------|-----------------|
| Personalization    | Moderate         | Moderate           | Low                  | Moderate        |
| AI Capabilities    | Moderate         | Low-Moderate       | Low                  | Low-Moderate    |
| Assessment Methods | Adaptive Quizzes | Practice Exercises | Assignment & Quizzes | Quizzes & Exams |
| Feedback Mechanism | Immediate        | Immediate          | Instructor-Based     | Mostly Delayed  |

### 2.7.6 Identified Limitations of Existing Systems

From the review, several limitations are identified across existing platforms:

Limited integration of fully AI-driven adaptive assessment systems

Weak real-time personalization during learning sessions

Lack of intelligent tutoring capabilities in most platforms

Feedback is often generic or delayed in many systems

Fragmentation between learning, assessment, and recommendation tools

### Conclusion

Existing educational platforms such as Duolingo, Khan Academy, Moodle, and Coursera provide valuable learning support but still lack fully integrated AI-powered adaptive assessment systems.

Most platforms offer partial personalization and limited real-time adaptation. These limitations highlight the need for a unified Interactive AI-Powered Study Assistant that combines adaptive assessment, intelligent tutoring, real-time feedback, and machine learning-based personalization into a single system for undergraduate students.

## 2.8 Theoretical Framework

The theoretical framework of this study is based on established learning theories that explain how students acquire, process, and retain knowledge in an adaptive learning environment. These theories provide a foundation for the design and implementation of the Interactive AI-Powered Study Assistant for Undergraduate Students using Adaptive

Assessment-Based Learning.

### **2.8.1 Constructivist Learning Theory**

Constructivist Learning Theory, proposed by Jean Piaget and further developed by Lev Vygotsky, emphasizes that learners construct knowledge actively rather than passively receiving information. Learning occurs when students interact with content, reflect on experiences, and build new understanding based on prior knowledge.

In the context of this study, the AI-powered study assistant supports constructivism by:

- Allowing learners to engage actively with quizzes and learning materials
- Providing interactive feedback that encourages reflection
- Adapting content based on student responses and understanding

Recent research shows that constructivist-based digital learning environments improve student engagement and knowledge retention by promoting active participation (Gligorea et al., 2023; Wang et al., 2024).

### **2.8.2 Self-Regulated Learning Theory**

Self-Regulated Learning (SRL) theory explains how learners take control of their own learning process through planning, monitoring, and evaluating their progress. According to this theory, successful learners set goals, apply strategies, and reflect on outcomes.

The proposed system supports self-regulated learning by:

- Allowing students to learn at their own pace
- Providing progress tracking dashboards
- Offering personalized feedback and recommendations
- Encouraging learners to identify and improve weak areas

AI-based learning systems enhance SRL by giving learners real-time insights into their performance and learning behavior (Hoogers et al., 2025).

### **2.8.3 Cognitive Load Theory**

Cognitive Load Theory, developed by John Sweller, explains that human working memory has limited capacity, and learning is most effective when instructional materials are designed to avoid overload.

In this study, the theory is applied by:

- Presenting learning materials in structured and simplified formats
- Adjusting difficulty levels through adaptive assessment
- Providing step-by-step explanations for complex topics
- Reducing unnecessary or repetitive information

Adaptive systems help reduce cognitive overload by ensuring that learners only receive content appropriate to their current level of understanding (Wang et al., 2024).

### **2.8.4 Adaptive Learning Theory**

Adaptive Learning Theory focuses on the idea that instruction should continuously adjust based on learner performance, behavior, and feedback. It forms the core foundation of AI-driven educational systems.

In the proposed system, adaptive learning is implemented through:

- Continuous assessment of student performance
- Dynamic adjustment of question difficulty
- Personalized learning pathways
- AI-based recommendation of learning resources

Recent studies confirm that adaptive learning systems significantly improve academic performance by tailoring instruction to individual learner needs (Boulhrir et al., 2026; Yuensook et al., 2024).

### **Conclusion of Theoretical Framework**

The combination of Constructivist Learning Theory, Self-Regulated Learning Theory, Cognitive Load Theory, and Adaptive Learning Theory provides a strong foundation for this study. These theories collectively support the development of an intelligent system that promotes active learning, learner independence, reduced cognitive overload, and personalized education. This theoretical foundation justifies the design of an Interactive AI-Powered Study Assistant using Adaptive Assessment-Based Learning.

## **2.9 Conceptual Framework**

The conceptual framework of this study illustrates how the Interactive AI-Powered Study Assistant processes inputs from learners and learning resources, applies intelligent

processing mechanisms, and generates personalized educational outputs. It explains the relationship between student data, system processes, and adaptive learning outcomes within the proposed system.

## **2.9.1 Inputs**

The system relies on three main inputs to function effectively:

### **1. Student Profile**

This includes learner-specific data such as:

- Registration details
- Academic level
- Learning preferences
- Historical performance

The student profile helps the system understand individual learner characteristics and personalize learning experiences.

### **2. Learning Materials**

These are educational resources provided within the system, including:

- Lecture notes
- Topic summaries
- Multimedia content
- Reference materials

Learning materials form the core knowledge base used by the system to support instruction.

### **3. Assessment Questions**

These are structured questions used to evaluate student understanding, including:

- Diagnostic questions
- Formative quizzes
- Practice exercises
- Summative tests

Assessment questions help measure learner progress and identify knowledge gaps.

## **2.9.2 Processes**

The system applies intelligent processing techniques to transform inputs into meaningful learning outputs:

### **1. Performance Analysis**

The system analyzes student responses to determine:

- Accuracy of answers
- Completion time
- Performance trends

This helps evaluate student understanding and engagement.

### **2. Knowledge Gap Detection**

The system identifies areas where the student lacks understanding by:

- Comparing performance across topics
- Detecting incorrect or repeated mistakes
- Highlighting weak subject areas

This enables targeted learning interventions.

### **3. Adaptive Recommendation Engine**

The recommendation engine uses AI techniques to:

- Suggest relevant learning materials
- Recommend practice questions
- Adjust learning difficulty
- Generate personalized study plans

This ensures that learning is tailored to each student's needs.

## **2.9.3 Outputs**

The system produces intelligent and personalized outputs that support learning:

### **1. Personalized Learning Path**

A structured and adaptive learning route tailored to each student based on their

performance and knowledge level.

## 2. Adaptive Quizzes

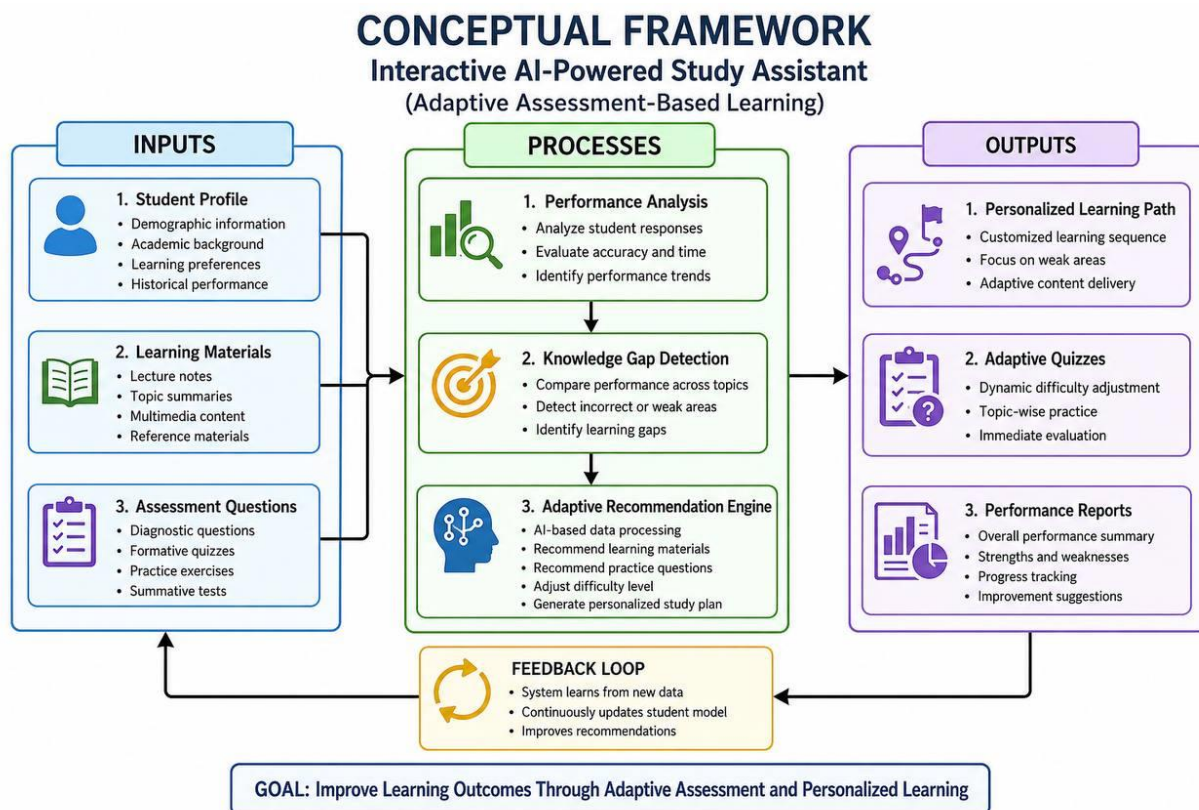
Automatically generated quizzes that adjust in difficulty depending on student responses and progress.

## 3. Performance Reports

Detailed reports that provide insights into:

Student strengths and weaknesses  
Progress over time  
Areas requiring improvement

These reports help both students and educators track academic development.



### 2.9.4 Summary of the Conceptual Framework

The conceptual framework demonstrates that the system operates as a continuous cycle of input, processing, and output. Student data and learning materials are analyzed

through performance evaluation and knowledge gap detection, which then drive the recommendation engine. The resulting outputs personalized learning paths, adaptive quizzes, and performance reports enhance student learning outcomes and support adaptive assessment-based learning.

## **Conclusion**

This conceptual framework provides the foundation for the design and implementation of the proposed AI-powered study assistant. It ensures that learning is dynamic, personalized, and continuously improved through adaptive assessment and intelligent feedback mechanisms.

## **2.10 Research Gap**

Despite the rapid advancement of Artificial Intelligence (AI), Intelligent Tutoring Systems (ITS), and adaptive learning technologies in education, existing systems still present several limitations that reduce their effectiveness in supporting personalized learning for undergraduate students.

### **1. Limited Personalization**

Most existing educational platforms provide only partial personalization. Learning content is often standardized or semi-adaptive, meaning it does not fully adjust to individual student needs, learning speed, or knowledge level in real time. As a result, students with different abilities often receive similar learning experiences, which reduces learning efficiency (Gligorea et al., 2023; Wang et al., 2024).

### **2. Lack of Adaptive Assessments**

Many current systems rely on fixed or static assessment methods where all students answer the same set of questions regardless of performance level. This limits the system's ability to accurately measure student ability and provide appropriate challenge levels. Adaptive assessment mechanisms such as dynamic difficulty adjustment is still not fully implemented in most learning platforms (Hoogers et al., 2025).

### **3. Inadequate Feedback Mechanisms**

Feedback in many existing systems is often delayed, generic, or not context-aware. Students may receive correct/incorrect responses without detailed explanations or guidance for improvement. Zhan et al.,(2025) define inadequate feedback as a primary issue in higher education where students disengage from learning process due to contextual constraints such as large classes and limited curriculum space, resulting in feedback that fails to achieve its expected effects on student learning improvement.

#### **4. Poor Integration of AI-Driven Recommendations**

Although some platforms include recommendation features, many lack deep integration of AI-based recommendation engines that continuously analyze student performance and update learning paths dynamically. Recommendations are often static or based on simple rules rather than intelligent machine learning models (Boulhrir et al., 2026).

#### **5. Lack of Unified Intelligent Learning Systems**

Akgun, S., & Greenhow, C. (2025) identify that while AI-powered educational tools have demonstrated effectiveness in "personalized learning pathways, adaptive assessments, intelligent tutoring systems, and support for diverse learning needs," most implementations remain siloed rather than integrated, with "challenges like data quality, ethical considerations, and institutional resistance" preventing the development of unified systems that combine multiple intelligent functions into a single cohesive platform for learners.

### **Conclusion of Research Gap**

The literature clearly shows that while AI has significantly improved educational systems, there is still a gap in fully integrated, adaptive, and intelligent learning platforms. These limitations justify the need for an Interactive AI-Powered Study Assistant that combines adaptive assessment-based learning, personalized learning pathways, real-time feedback, and AI-driven recommendations to improve undergraduate students' academic performance.

## CHAPTER THREE: METHODOLOGY

### 3.1 Research Design

The research design for the Smart Study Assistant combines developmental and experimental approaches. The developmental component focuses on the systematic creation of the software artifact, while the experimental component evaluates its effectiveness in improving student learning outcomes.

#### 3.1.1 Developmental Research Design

Following software engineering research principles, key activities included:

- **Requirements gathering** – questionnaires, interviews, and observation to capture user needs.
- **Architectural design** – five-layer architecture (UI, Application, AI Engine, Adaptive Assessment Engine, Database).
- **Iterative implementation** – Agile sprints using PHP, MySQL, HTML/CSS/JS, and a Python microservice for ML recommendations.
- **Documentation** – database schemas, algorithms, and user guides.

The outcome is a deployable Smart Study Assistant featuring user authentication, an AI chatbot, document analysis, quiz generation, flashcards, an OCR-based notes converter, and a recommendation engine.

#### 3.1.2 Experimental Research Design

A one-group pretest–posttest design measures the system's impact on learning. Students (20, ITDS undergraduates at UENR, convenience sampling) took a standard quiz before the intervention, used the system for two weeks, then completed a parallel posttest. Mean scores were compared using a paired t-test; usability was measured via the System Usability Scale (SUS). The hypothesis is that posttest scores will be significantly higher, indicating that the system enhances learning.

### 3.2 System Development Methodology

The project followed the Agile (Scrum) methodology to accommodate changing requirements, iterative delivery, and continuous feedback.

#### 3.2.1 Requirements Gathering

Stakeholders (ITDS undergraduates) were identified, and data was collected via questionnaires, semi-structured interviews, observation sessions, and analysis of academic records. Requirements were prioritised using the MoSCoW method (Must have, Should have, Could have, Won't have), ensuring that critical features; student login, quiz generation, AI chatbot, document upload were delivered first. The outcome was a

Software Requirements Specification (SRS) listing functional and non-functional requirements.

### **3.2.2 System Design**

A five-layer architecture was designed alongside an ER diagram (seven core entities), AI algorithm design (rule-based adaptation and Random Forest), UI wireframes (Figma), and a finalised technology stack (PHP, MySQL, HTML/CSS/JS/Bootstrap, Python/Scikit-learn). All designs were reviewed before proceeding to development.

### **3.2.3 Development**

Development was executed in two-week sprints, each delivering a usable increment:

- Sprint 1 – User authentication (sign-up/login) and session management.
- Sprint 2 – AI chatbot integration (Mistral 7B API).
- Sprint 3 – Document upload and text extraction (PDF, DOCX, PPTX).
- Sprint 4 – Quiz generator with topic, difficulty, and question-type selection; auto-marking.
- Sprint 5 – Flashcard generation, OCR image converter, dashboard enhancements.
- Sprint 6 – Performance tracking, adaptive difficulty logic, admin panel stub, and ML recommendation microservice.

Daily standups, continuous integration (Git), and pair programming were practised throughout.

### **3.2.4 Testing**

Each sprint concluded with unit testing (PHPUnit, Python unittest), integration testing, system testing against the SRS, and User Acceptance Testing (UAT) with five volunteer students. Defects were logged and resolved in subsequent sprints.

### **3.2.5 Deployment**

The system was deployed locally using XAMPP (Apache, PHP 8.0, MySQL 8.0) and a Python 3.9 virtual environment. The database was migrated via phpMyAdmin; API keys and upload limits were configured. Smoke tests confirmed operational readiness.

### **3.2.6 Evaluation**

Evaluation confirmed functional completeness of all Must-have requirements. Performance testing showed acceptable response times (quiz generation avg. 6 s, document extraction 2 s, recommendation API 0.5 s). The SUS questionnaire (n=20) returned an average score of 82.5, indicating good usability. The system was demonstrated to peers and the evaluation panel.

## **3.3 Requirements Analysis**

Requirements were gathered through questionnaires, interviews, observation, and academic records analysis (Section 3.5) and are categorised as functional and non-functional.

## Functional Requirements

| Requirement                          | Description                                                                                                                                |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Registration/Login                   | Students create accounts (email, username, phone) with hashed passwords and session tracking.                                              |
| Take Assessments                     | Customised quizzes by topic, question type, quantity (5–20), difficulty, and time limit. Auto-marks objective questions; AI grades theory. |
| View Recommendations                 | Post-assessment, the system displays personalised resource recommendations (PDFs, videos) stored in the database.                          |
| Track Performance                    | Dashboard shows score history over time, past results, and topics needing improvement.                                                     |
| Manage Courses<br>(Admin/lecturer)   | Admin can add, edit, or delete courses with title, description, and default difficulty.                                                    |
| Upload Resources<br>(Admin/lecturer) | Admin uploads PDFs or YouTube links tagged by topic, difficulty, and course for the recommendation engine.                                 |
| Monitor Progress<br>(Admin/lecturer) | Admin views student averages per course and detailed performance reports.                                                                  |
| Analyse Performance (AI)             | After each quiz, the AI computes percentage scores and identifies weak topics stored in performance_records.                               |
| Adapt Difficulty (AI)                | Rule-based logic adjusts next assessment difficulty based on the most recent score thresholds.                                             |
| Recommend Content (AI)               | Personalised resources are retrieved based on weak topics and difficulty level using rule-based and ML methods.                            |

## Non-Functional Requirements

| Requirement | How It Is Addressed                                                                                                                                          |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Security    | Passwords hashed via password_hash(); prepared statements prevent SQL injection; session management restricts unauthorised access.                           |
| Reliability | Error handling for AI API calls and file uploads; database transactions ensure data consistency.                                                             |
| Scalability | Three-tier architecture separates concerns; ML microservice (Flask) scales independently; database normalised to 3NF.                                        |
| Usability   | Clean UI with sidebar navigation, tooltips, and a help section. SUS score: 82.5.                                                                             |
| Performance | Asynchronous AI calls with loading animations; indexed foreign keys; efficient extraction libraries. Targets: quiz generation < 10 s, document upload < 5 s. |

### 3.4 System Architecture

The Smart Study Assistant follows a five-layer architecture that separates concerns, improves maintainability, and allows independent scaling of AI components. The diagram below shows the flow of the system's architecture:

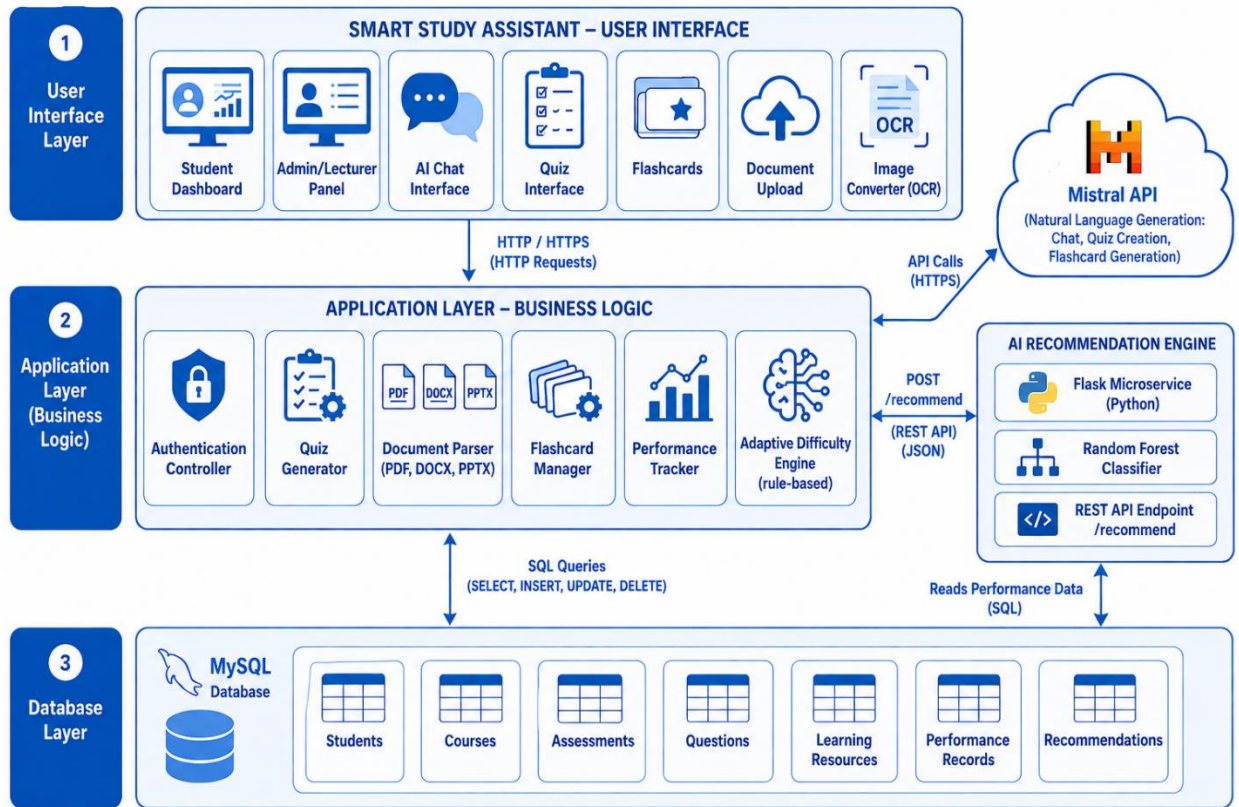


Figure 3.1: A Diagram showcasing the flow of the systems architecture from the UI, Application layer and Database layer.

## 1. User Interface Layer

Built with HTML5, CSS3, JavaScript, and Bootstrap 5.3, this layer renders the student dashboard, admin panel, chat interface, quiz form, flashcard page, document upload, and image converter. It handles user input and triggers AJAX calls to the application layer.

## 2. Application Layer

PHP and Python/Flask process frontend requests, enforce business rules (e.g., role-based access), interact with the database, and call AI services.

## 3. AI Recommendation Engine

A Python/Flask/Scikit-learn microservice exposes a REST endpoint (/recommend). A pre-trained Random Forest model receives a student's last score and difficulty level and returns the recommended resource difficulty and resource IDs. The engine is stateless and independently deployable.

## 4. Adaptive Assessment Engine

PHP rule-based logic analyses post-quiz scores against thresholds (<50%, 50–80%, >80%) and stores the recommended difficulty for the student's next quiz in the session or database, pre-selecting it on the quiz creation form.

## 5. Database Layer

MySQL provides persistent storage for all entities (Students, Courses, Assessments, Questions, Learning Resources, Performance Records, Recommendations) with ACID compliance and indexed foreign keys for efficient queries.

### Data Flow – Taking a Quiz

- Student selects topic, difficulty, and question count → UI sends a POST request to the Application Layer.
- Application Layer calls the Mistral AI API to generate questions, which are displayed to the student.
- On submission, the Application Layer scores objective questions, sends theory answers to the AI for grading, and saves results to `performance_records`.
- The Adaptive Assessment Engine determines the next recommended difficulty; the AI Recommendation Engine returns resource suggestions; both are stored and displayed to the student.

## 3.5 Data Collection Methods

Four methods were employed to ensure the system meets real user needs. All data was collected ethically with informed consent, and records were anonymised.

| Method                                                                                                                                                     | Participants           | Outcome                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|----------------------------------------------------------------------------------------------------------------|
| Questionnaires – 15-question Likert-scale survey on study habits, assessment preferences, and desired AI features.<br><br>Google Form shared via WhatsApp. | 30 ITDS undergraduates | Identified high demand for AI chatbot (85%), quiz generator (90%), and performance tracking (70%).             |
| Semi-structured Interviews – 15–20 min sessions on study challenges, digital tool use, and AI expectations.                                                | 10 ITDS students       | Revealed need for personalised feedback and adaptive difficulty requirements.                                  |
| Observation – 2-hour library study sessions, noting use of existing platforms (ChatGPT, YouTube, past questions).                                          | 6 students             | Showed students switch between multiple tabs, inspiring the all-in-one dashboard and document upload features. |

| Method                                                                                       | Participants           | Outcome                                                                                   |
|----------------------------------------------------------------------------------------------|------------------------|-------------------------------------------------------------------------------------------|
| Academic Records – Anonymised exam scores from Database Systems and Web Development courses. | 120 anonymised records | Confirmed wide score distribution (30%–95%), justifying the need for adaptive difficulty. |

### 3.6 Database Design

MySQL was selected for data integrity and complex query support. The schema follows Third Normal Form (3NF) to minimise redundancy and is composed of seven core entities.

Key relationships: Students → Performance Records (one-to-many); Courses → Assessments (one-to-many); Assessments → Questions (one-to-many); Students → Recommendations (one-to-many). Indexes are applied on all foreign keys to optimise dashboard and recommendation queries.

| Entity              | Key Attributes                                                                               | Description                                                                        |
|---------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Students            | student_id (PK),<br>username, email,<br>password_hash, role                                  | User accounts. role determines access rights (student/admin).                      |
| Courses             | course_id (PK),<br>course_name, description,<br>difficulty                                   | Academic subjects linked to assessments and resources.                             |
| Assessments         | assessment_id (PK),<br>course_id (FK), type,<br>difficulty, time_limit                       | Quiz/exam templates; multiple per course.                                          |
| Questions           | question_id (PK),<br>assessment_id (FK),<br>question_text, options<br>(JSON), correct_answer | Individual questions with JSON options for objectives.                             |
| Learning Resources  | resource_id (PK), title,<br>type, url, topic, difficulty                                     | Admin/lecturers-uploaded PDFs and YouTube links used by the recommendation engine. |
| Performance Records | record_id (PK), student_id<br>(FK), assessment_id (FK),<br>score, answers_json,<br>taken_at  | Logs every quiz attempt for tracking and trend analysis.                           |

| Entity          | Key Attributes                                                     | Description                                                  |
|-----------------|--------------------------------------------------------------------|--------------------------------------------------------------|
| Recommendations | rec_id (PK), student_id (FK), resource_id (FK), reason, created_at | Stored recommendations for re-display without recomputation. |

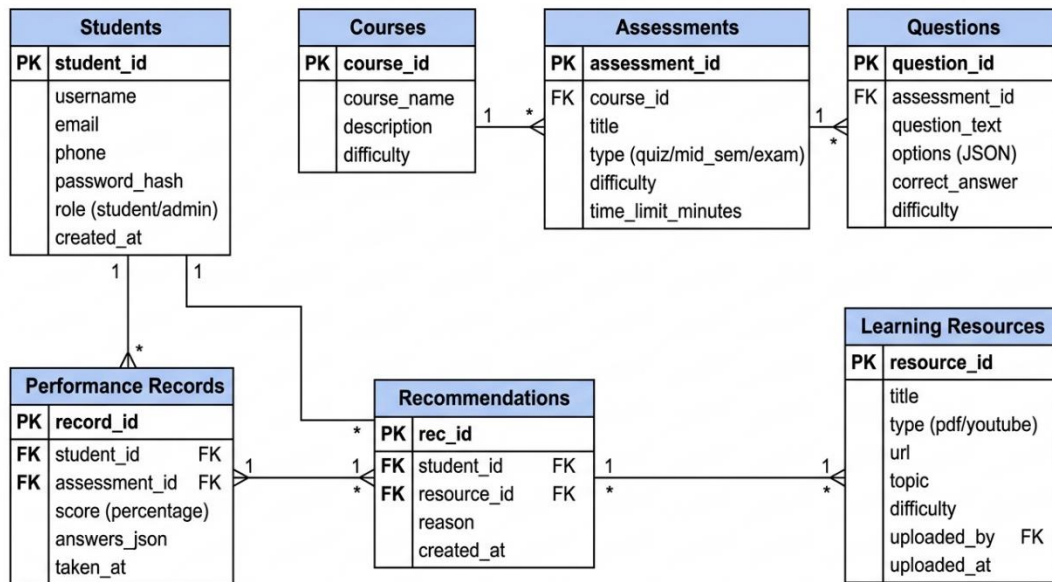


Figure 3.2: A UML Diagram showing the entities and their relations in the database

## 3.7 AI Techniques

The system employs two levels of AI: rule-based adaptation for immediate, transparent responses, and optional machine learning models for data-driven personalisation.

### Rule-Based Adaptation

Implemented directly in PHP, this approach uses score thresholds to adjust difficulty and resource recommendations without requiring historical data:

```

If score < 50      → next_difficulty = 'easy',    resource_level = 'basic'
If 50 ≤ score ≤ 80 → next_difficulty = 'medium',  resource_level =
'intermediate'
If score > 80     → next_difficulty = 'hard',    resource_level = 'advanced'
  
```

After each quiz, the recommended difficulty is stored in `$_SESSION['next_difficulty']` and pre-selected on the quiz creation form.

## Machine Learning Model: Random Forest Classifier:

For more sophisticated personalisation, the system includes a **Python-based microservice** using **Scikit-learn's Random Forest Classifier**. Random Forest was chosen because it handles non-linear relationships, resists overfitting with small datasets, and provides feature importance.

- **Input features (3):**
  - last\_score – the student's most recent quiz score (0–100)
  - last\_difficulty – encoded as 0 (easy), 1 (medium), 2 (hard)
  - times\_taken – total number of quizzes the student has previously taken
- **Target output:**
  - recommended\_difficulty – 0 (easy), 1 (medium), 2 (hard)
- **Training data:** Initially seeded with 200 synthetic records based on typical student performance (e.g., low scores → easy, high scores → hard). As real performance\_records accumulate, the model is retrained weekly.
- **Why Random Forest?** It outputs feature importance (confirming that last\_score is the strongest predictor), works well with tabular data, and is easy to deploy as a lightweight microservice.

### Implementation details:

- **Libraries:** Scikit-learn (RandomForestClassifier), Pandas, NumPy, Flask.
- **Microservice endpoint:** POST /recommend expects JSON {"score":75, "last\_difficulty":1, "times\_taken":5} and returns {"recommended\_difficulty":2, "resource\_ids":[12,34]}.
- **Fallback:** If the Python microservice is unavailable or the model has not yet been trained, the system automatically falls back to the rule-based method described above.

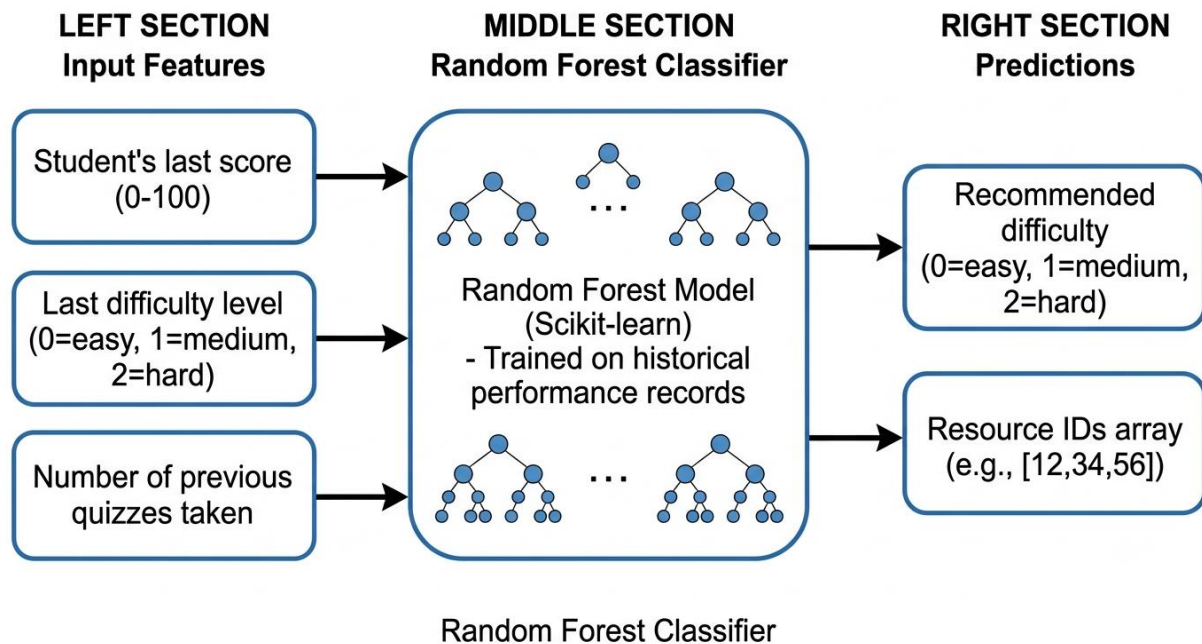


Figure 3.3: A Diagram showing how the proposed recommendation system will be implemented using random forest model

### 3.8 Tools and Technologies

| Category     | Technology / Version                                           | Justification                                                                                                               |
|--------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Frontend     | HTML5, CSS3, JavaScript (ES6), Bootstrap 5.3                   | Standard web technologies for responsive, dynamic interfaces. Bootstrap accelerates development with pre-styled components. |
| Backend      | PHP 8.0 (main logic), Python 3.9 / Flask 2.3 (AI microservice) | PHP for core application logic; Python/Flask for the lightweight, scalable recommendation API.                              |
| Database     | MySQL 8.0                                                      | Open-source, reliable, supports JSON data types for quiz answers, integrates well with PHP and Python.                      |
| AI Libraries | Scikit-learn 1.3, Pandas 2.0, NumPy 1.24                       | Scikit-learn provides all required ML algorithms out of the box; Pandas and NumPy support data manipulation.                |

| Category          | Technology / Version       | Justification                                                                                       |
|-------------------|----------------------------|-----------------------------------------------------------------------------------------------------|
| Development Tools | XAMPP 8.0, Git 2.40, Figma | XAMPP for local development; Git for version control; Figma for UI mockups prior to implementation. |

Integration overview: The frontend communicates with the PHP backend via AJAX; PHP calls the Mistral external AI API for natural language generation and the local Flask microservice for recommendations; Flask uses Scikit-learn models trained on MySQL data.

### 3.9 System Testing

Testing was conducted throughout the Agile development lifecycle across four levels to ensure correctness, reliability, and user satisfaction.

#### 1. Unit Testing

Individual PHP functions and Python API endpoints were tested in isolation using PHPUnit and Python's unit test. Sample cases confirmed correct password hashing, PDF text extraction, rule-based difficulty output, and Flask recommendation responses. All tests passed before integration.

#### 2. Integration Testing

Module interactions were validated across key flows: Login → Session → Dashboard; Quiz Submission → Score Storage → Recommendation Call; Admin Upload → Resource Listing. All integration points worked correctly after resolving initial JSON formatting mismatches between PHP and Flask.

#### 3. System Testing

The fully integrated system was tested against all 15 functional requirements using a predefined test plan with valid and invalid inputs. Selected results:

| Test ID | Requirement           | Action                       | Result                                |
|---------|-----------------------|------------------------------|---------------------------------------|
| SYS01   | Student registration  | Submit form with new email   | Pass – account created                |
| SYS02   | Duplicate email       | Register with existing email | Pass – error message shown            |
| SYS03   | Generate quiz         | Select topic, 10 questions   | Pass – quiz displayed with timer      |
| SYS04   | Adaptive difficulty   | Score 30% on a quiz          | Pass – next difficulty preset to Easy |
| SYS05   | Admin resource upload | Upload PDF with metadata     | Pass – resource in DB table           |

Load testing with Apache JMeter (50 concurrent users) returned average response times of 1.2 s (login), 6.5 s (quiz generation), and 4.8 s (chat) – within acceptable limits.

#### **4. User Acceptance Testing (UAT)**

Twenty ITDS students who had not participated in development used the system freely for seven days, then completed five scenario-based tasks (register, upload document, chat with AI, generate quiz, view recommendations) and the SUS questionnaire.

- Task completion rate: 80% without assistance.
- Bugs found: 3 minor issues (e.g., timer reset on tab switch) – all resolved.
- Average SUS score: 82.5 (Grade A – excellent usability).
- Qualitative feedback: Students valued the document upload and adaptive difficulty features; some requested a mobile app.

The system passed UAT and was deemed ready for local demonstration deployment.

## CHAPTER FOUR: SYSTEM DEVELOPMENT AND IMPLEMENTATION

### 4.1 System Analysis

#### 4.1.1 Existing System Analysis

Before developing the Smart Study Assistant, an analysis of existing study methods and tools used by ITDS students was conducted. Most students relied on a combination of general-purpose tools:

- **Search engines (Google)** and **YouTube** for finding explanations and tutorials.
- **Messaging platforms (WhatsApp, Telegram)** to share notes and past questions.
- **Manual flashcards** (physical or using apps like Anki) for revision.
- **Microsoft Word or PDF readers** for accessing lecture notes.

#### Identified limitations of the existing approach:

- **Fragmentation** – students switch between multiple applications (browser, chat, document viewer) during a single study session.
- **No personalisation** – resources are not tailored to individual performance or knowledge gaps.
- **No automatic assessment** – students must manually create or search for practice questions; no instant feedback.
- **No performance tracking** – students cannot easily see their progress over time or identify weak topics.

These gaps motivated the development of an integrated, AI-powered study assistant.

#### 4.1.2 Proposed System Analysis

The proposed Smart Study Assistant addresses the limitations by providing a single-platform solution with the following key improvements:

- **All-in-one dashboard** – chat, quiz, document upload, flashcards, and image converter are accessible from a unified sidebar menu.
- **AI-driven personalisation** – rule-based and machine learning (Random Forest) adaptation adjusts difficulty and recommends resources based on individual performance.
- **Automated assessment** – students generate custom quizzes on any topic; objective questions are auto-marked, and theory answers are graded by the AI.
- **Performance tracking** – every quiz attempt is stored in the database, and a dashboard displays historical scores and identifies weak areas.
- **Admin support** – lecturers can upload learning resources and monitor student progress through an admin panel.

The proposed system is web-based, accessible via a local server (XAMPP), and uses PHP for the backend, MySQL for the database, and an external AI API (Mistral) for natural language generation. A Python microservice with Scikit-learn provides personalised recommendations.

### 4.2 System Design

This section presents the Unified Modelling Language (UML) diagrams that illustrate the static structure and dynamic behaviour of the Smart Study Assistant.

#### 4.2.1 Use Case Diagram

The use case diagram identifies the two actors (Student and Admin/Lecturer) and their interactions with the system.

- **Student** can: register/login, take assessments, generate flashcards, upload documents, chat with AI, convert image notes (OCR), view personalised recommendations, and track performance.
- **Admin/Lecturer** can: manage courses, upload learning resources, and monitor student progress.

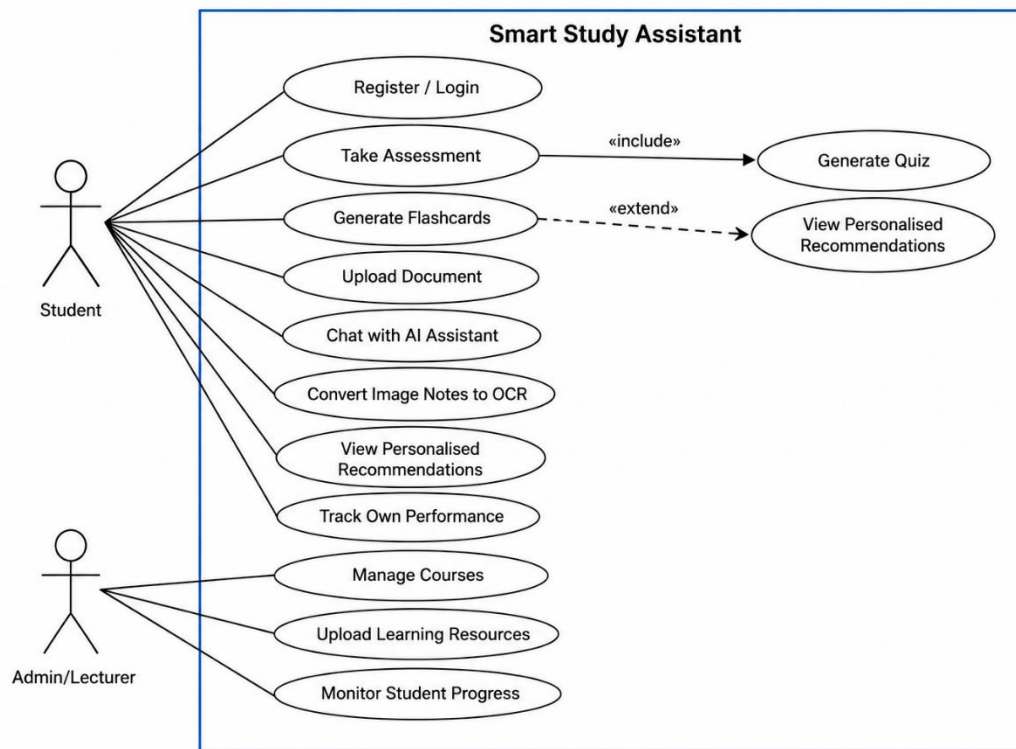


Figure 4.1: Use Case Diagram showing interactions between the Student and Admin/Lecturer actors with the Smart Study Assistant system

### 4.2.2 Activity Diagram

The activity diagram below shows the flow of actions for a student taking a quiz. It includes: selecting topic and difficulty, generating questions, answering, submitting, automatic marking, storing score, and receiving recommendations.

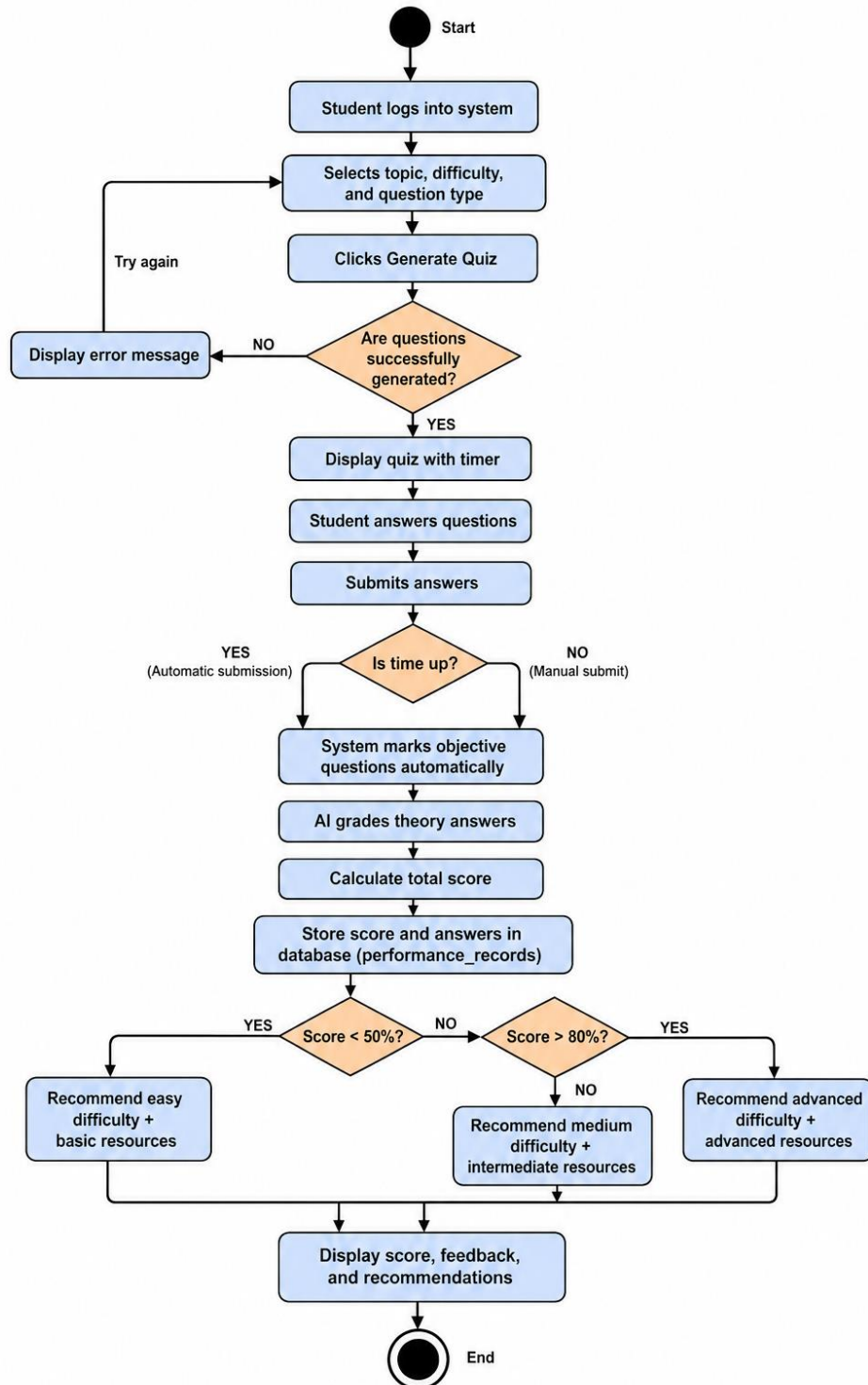


Figure 4.2: Activity Diagram illustrating the step-by-step flow of actions when a student takes a quiz, from topic selection through to receiving recommendations

### 4.2.3 Sequence Diagram

The sequence diagram illustrates the interaction between objects when a student requests a quiz: the frontend sends parameters to the PHP backend, which calls the external AI API (Mistral) to generate questions, returns them to the frontend, and later receives answers for grading.

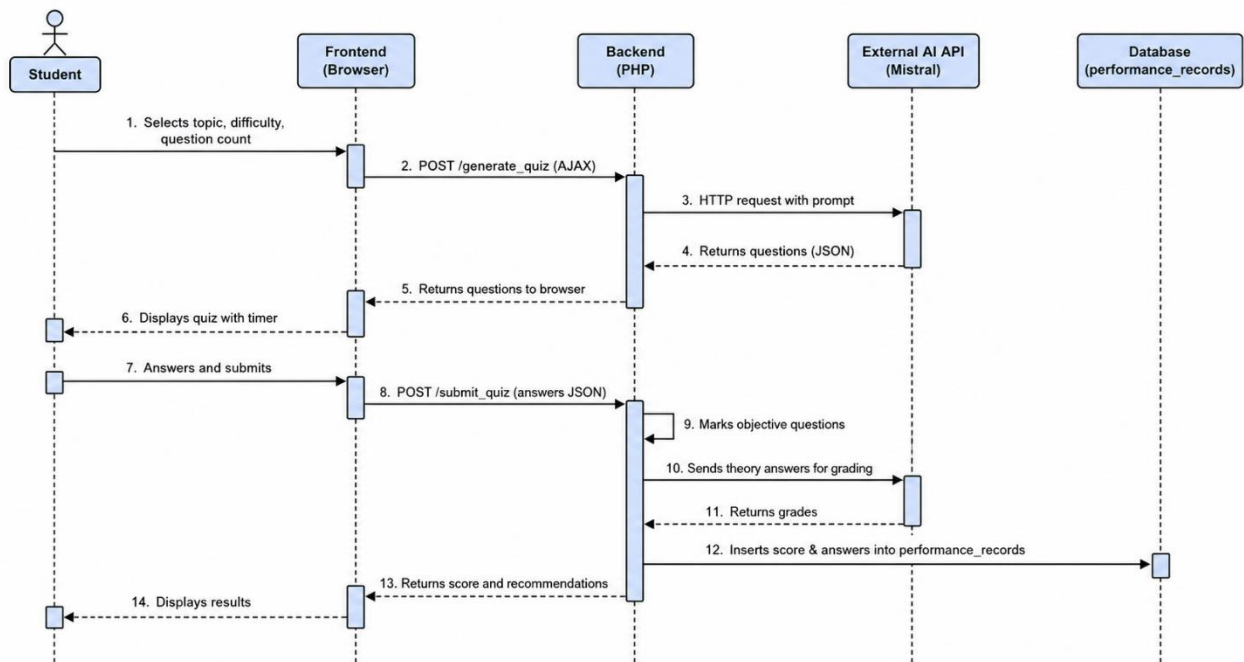


Figure 4.3: Sequence Diagram showing the message flow between the frontend, PHP backend, and Mistral AI API during quiz generation and submission

### 4.2.4 Class Diagram

The class diagram shows the main classes in the system (e.g., User, Assessment, Question, PerformanceRecord, Resource) and their attributes and methods. Relationships such as inheritance (if any) and associations (one-to-many) are depicted.

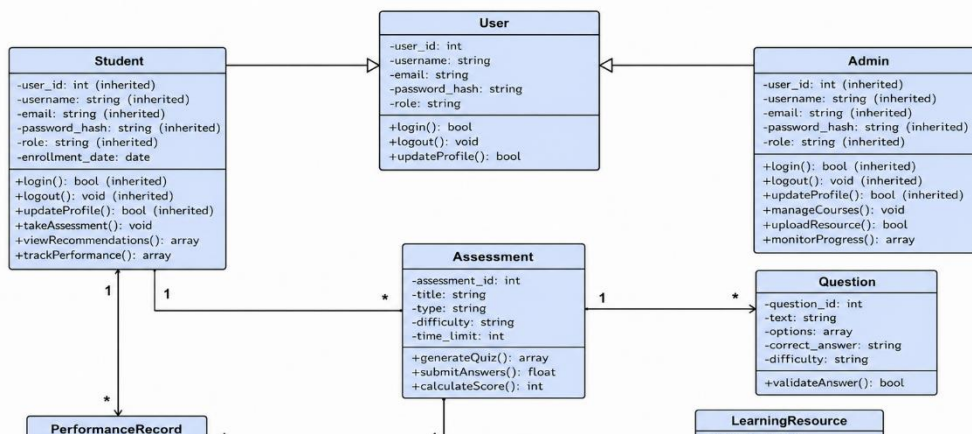


Figure 4.4: Class Diagram depicting the main system classes (User, Assessment, Question, PerformanceRecord, Resource), their attributes, methods, and associations

## 4.3 User Interface Design

The user interface was designed to be intuitive, responsive, and consistent with a dark/light mode toggle. Below are screenshots of the key pages.

### 4.3.1 Login Page

The login page allows existing users to sign in with their email and password. New users can switch to the registration form. Input validation (email format, password strength) is performed client-side and server-side.

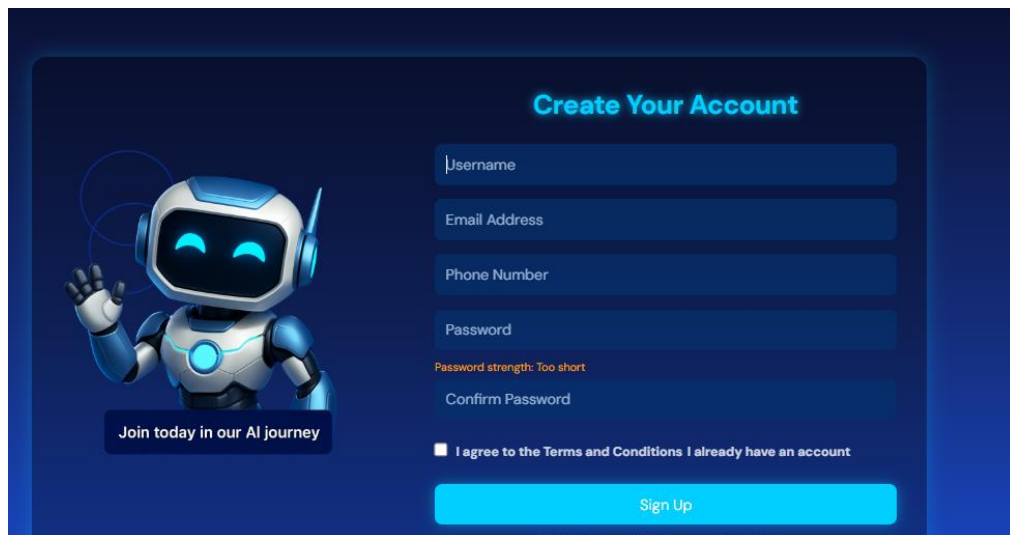


Figure 4.5: Screenshot of the Login Page, showing the email/password input fields, client-side validation, and link to the registration form

### 4.3.2 Dashboard

After login, the student sees the main dashboard. The sidebar provides access to all features: Chat, Quiz, Flashcards, Document Upload, Image Converter, and (for admins) Admin Panel. The dashboard also displays recent activity and quick tips.

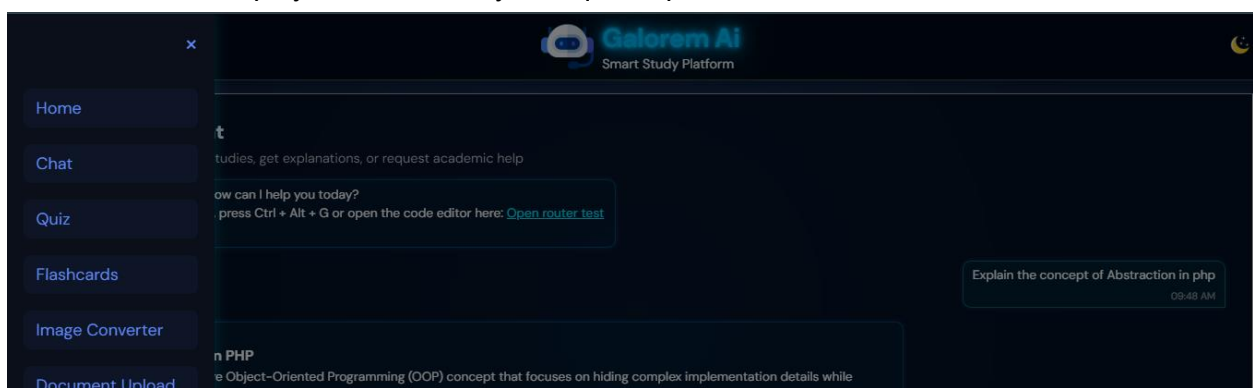
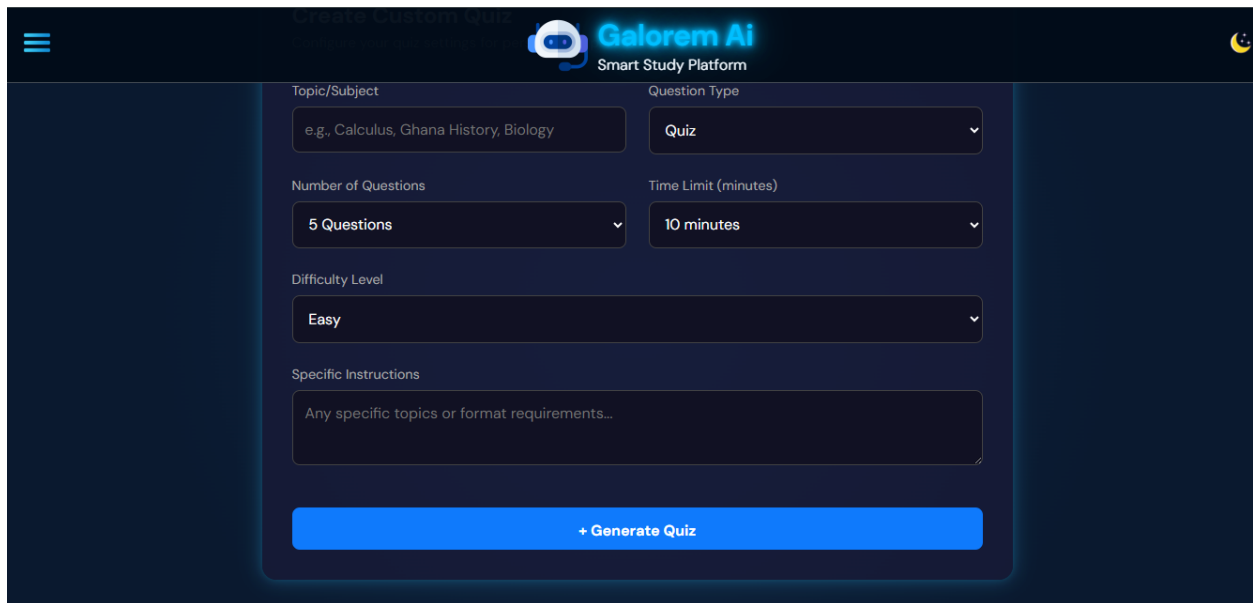


Figure 4.6: Screenshot of the Student Dashboard displaying the sidebar navigation menu and quick-access links to all system features

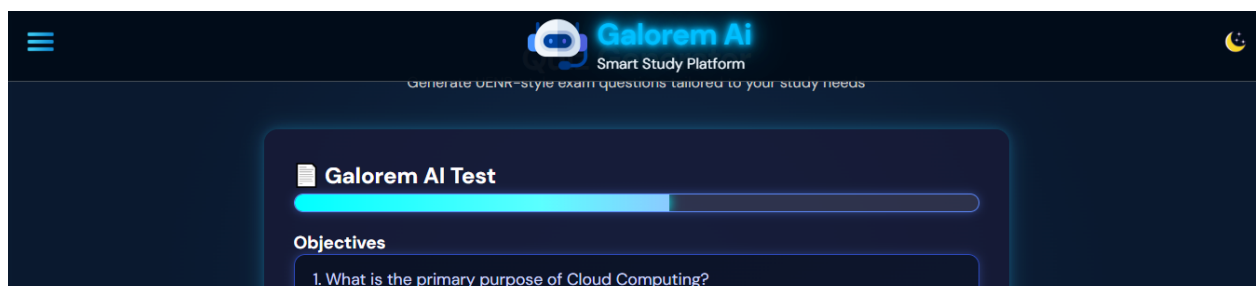
### 4.3.3 Assessment Interface

The quiz generation form lets students enter a topic, select question type (quiz, mid-sem, exam), number of questions (5-20), difficulty (easy/medium/hard), and time limit. After generation, questions are presented with a timer and auto-submission.



The screenshot shows the 'Quiz Generation Form' interface. At the top, there is a header with a hamburger menu icon on the left, the 'Galorem Ai Smart Study Platform' logo in the center, and a moon icon on the right. The form itself is a dark blue card with several input fields and a large blue button at the bottom. The fields are: 'Topic/Subject' with a text input containing 'e.g., Calculus, Ghana History, Biology'; 'Question Type' with a dropdown menu set to 'Quiz'; 'Number of Questions' with a dropdown menu set to '5 Questions'; 'Time Limit (minutes)' with a dropdown menu set to '10 minutes'; 'Difficulty Level' with a dropdown menu set to 'Easy'; and 'Specific Instructions' with a text area containing 'Any specific topics or format requirements...'. A large blue button at the bottom is labeled '+ Generate Quiz'.

Figure 4.7: Screenshot of the Quiz Generation Form where students select topic, question type, quantity, difficulty, and time limit



The screenshot shows the 'Galorem AI Test' interface. At the top, there is a header with a hamburger menu icon on the left, the 'Galorem Ai Smart Study Platform' logo in the center, and a moon icon on the right. Below the header, there is a blue banner with the text 'Generate GCIR-style exam questions tailored to your study needs'. The main content area is a dark blue card with a title 'Galorem AI Test' and a progress bar. Below the progress bar, there is a section titled 'Objectives' with a list of questions, including '1. What is the primary purpose of Cloud Computing?'.

Figure 4.8: Screenshot of an Active Quiz session showing generated questions, answer options, and the countdown timer

#### 4.3.4 Recommendation Page

After completing a quiz, the system displays personalised recommendations: YouTube videos and PDFs related to weak topics, and a suggested difficulty for the next quiz. Recommendations are also accessible from the dashboard.

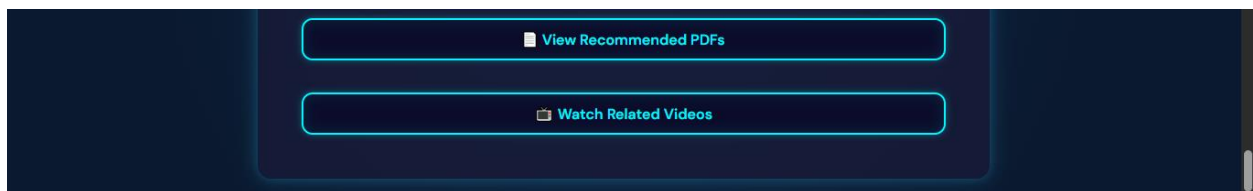


Figure 4.9: Screenshot of the Recommendation Page displaying personalised YouTube videos and PDF resources matched to the student's weak topics and difficulty level

#### 4.3.5 Performance Analytics Page

The performance analytics page shows a line graph of the student's scores over time, a breakdown of correct/incorrect answers per topic, and a list of past quiz attempts with scores.



Figure 4.10: Screenshot of the Performance Analytics Dashboard showing the score history line graph, topic breakdown, and list of past quiz attempts

## 4.4 System Implementation

The system was implemented using the following modules, each described below with key code snippets (or references to code files).

### 4.4.1 User Management Module

#### Authentication:

The registration and login system uses PHP sessions. Passwords are hashed using `password_hash()`. Upon successful login, `$_SESSION['user_id']` and `$_SESSION['role']` are stored.

php

// login.php (excerpt)

```
session_start();
$query = "SELECT * FROM users WHERE email='$email'";
$result = mysqli_query($conn, $query);
if (mysqli_num_rows($result) > 0) {
    $user = mysqli_fetch_assoc($result);
    if (password_verify($password, $user['password'])) {
        $_SESSION['user_id'] = $user['id'];
        $_SESSION['role'] = $user['role'];
        echo json_encode(['status' => 'success']);
    }
}
```

#### User

Students can view and edit their profile (username, email, phone). Profile data is stored in the users table.

#### Profiles:

KA

Calvin Akiweley

calvinakiweley@gmail.com

Student

Member Since

Jan 2025

Quizzes Taken

24

Avg. Score

72%

Flashcards Made

47

Study Streak

7 days

ACCOUNT INFORMATION

USERNAME

CalvinAkiweley

Edit

EMAIL ADDRESS

calvinakiweley@gmail.com

Edit

PHONE NUMBER

+233 55 385 9912

Edit

CHANGE PASSWORD

Figure 4.11: Screenshot of the User Profile Page where students can view and edit their username, email, and phone number

#### 4.4.2 Assessment Module

##### Question

Questions are generated on-the-fly by the external AI API (Mistral) based on the student's topic and difficulty. There is no persistent question bank; each quiz is dynamically created. However, a questions table exists for future extensibility.

##### Bank:

##### Quiz

The frontend sends an AJAX POST request to generate\_quiz.php, which calls the Mistral API with a structured prompt. The AI returns JSON containing objective and theory questions. The frontend renders the questions and starts the timer.

##### Generation:

**javascript**

```
// quiz.js (excerpt)

const response = await queryAI(prompt);

const data = JSON.parse(response);

renderQuestions(data);

startTimer(timeInMinutes * 60);
```

#### 4.4.3 Adaptive Learning Module

**php**

```
if ($score < 50) {

    $_SESSION['next_difficulty'] = 'easy';

} elseif ($score <= 80) {

    $_SESSION['next_difficulty'] = 'medium';

} else {

    $_SESSION['next_difficulty'] = 'hard';

}
```

##### Difficulty Adjustments (Rule-based):

After a quiz is submitted, the score is calculated. The PHP backend applies the following rule and stores the recommended difficulty in the session:

**php**

```
if ($score < 50) {

    $_SESSION['next_difficulty'] = 'easy';

}
```

```

} elseif ($score <= 80) {
    $_SESSION['next_difficulty'] = 'medium';
} else {
    $_SESSION['next_difficulty'] = 'hard';
}

```

#### 4.4.4 Recommendation Engine

##### Resource

Based on the student's score and weak topics (extracted from answers\_json), the system queries the learning\_resources table for matching materials. The results are displayed on the recommendation page.

**sql**

```

SELECT * FROM learning_resources
WHERE topic LIKE '%$weak_topic%' AND difficulty = '$recommended_level';

```

##### Recommendation:

##### Knowledge

After each quiz, the system analyses incorrect answers (using the AI to identify topics) and stores them in the performance\_records answers\_json field. The frontend highlights weak areas in the analytics dashboard.

##### Gap

##### Detection:

#### 4.5 Database Implementation

The database was implemented in MySQL according to the design in Section 3.6.

##### 4.5.1 Tables

The following tables were created: users (students), courses, assessments, questions, learning\_resources, performance\_records, recommendations. Example screenshot of the performance\_records table structure:

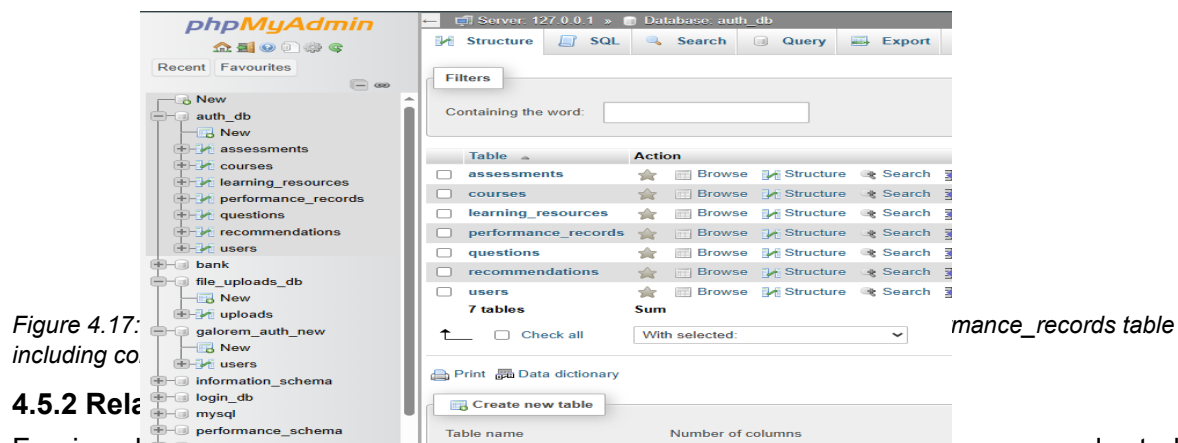


Figure 4.17:  
including co

##### 4.5.2 Rel

Foreign keys enforce referential integrity. For instance, performance\_records.student\_id references users.id. Indexes were added on foreign keys to improve query performance.

##### 4.5.3 Key Queries

Query to retrieve a student's score history:

**sql**

```

SELECT taken_at, score FROM performance_records
WHERE student_id = 1 ORDER BY taken_at ASC;

```

Query to get top recommended resources for a weak topic:

```
sql
```

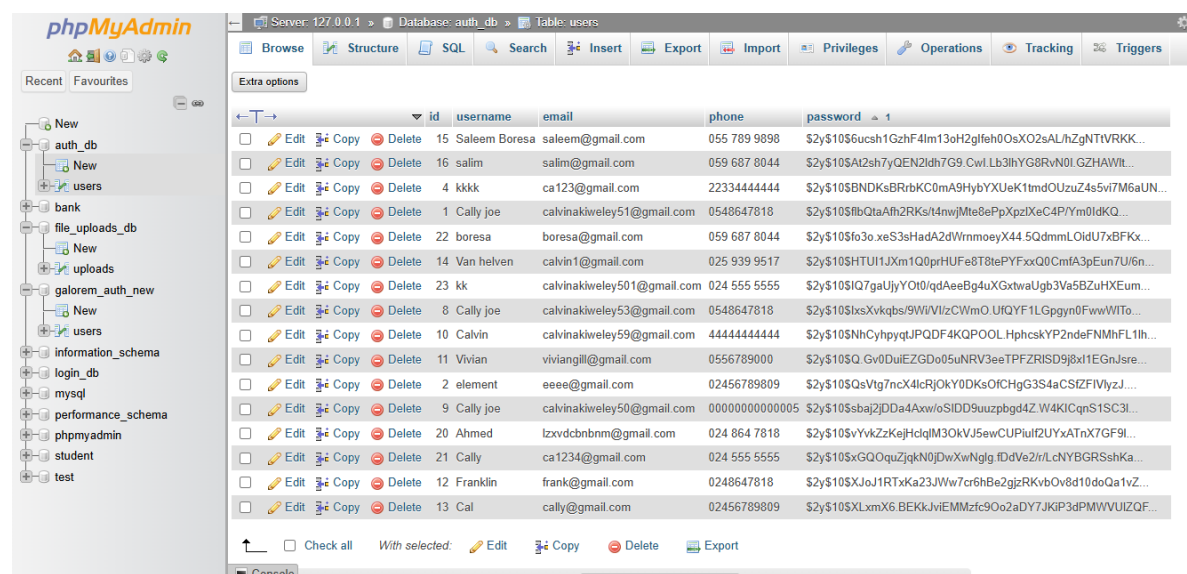
```
SELECT title, url, type FROM learning_resources
```

```
WHERE topic = 'SQL Joins' AND difficulty = 'easy' LIMIT 5;
```

## 4.6 System Security Measures

### 4.6.1 Password Encryption

User passwords are hashed using PHP's `password_hash()` with the default BCrypt algorithm. Plaintext passwords are never stored. Verification uses `password_verify()`.



The screenshot shows the phpMyAdmin interface with the 'users' table selected. The table contains 15 rows of user data. The 'password' column shows BCrypt hashes, confirming that no plaintext passwords are stored.

|                          | id | username      | email                       | phone          | password                                                 |
|--------------------------|----|---------------|-----------------------------|----------------|----------------------------------------------------------|
| <input type="checkbox"/> | 15 | Saleem Boresa | saleem@gmail.com            | 055 789 9898   | \$2y\$10\$Gucsh1GzhF4lm13oH2glfh0OosXO2sALhZgNTVRKK...   |
| <input type="checkbox"/> | 16 | salim         | salim@gmail.com             | 059 687 8044   | \$2y\$10\$A12sh7yQEN2ldh7G9.CwL.Lb3lHYG8RvN0I.GZHAWlt... |
| <input type="checkbox"/> | 4  | kkkk          | ca123@gmail.com             | 223344444444   | \$2y\$10\$BNDKsBRrbKC0mA9HybYXUeK1tmdOUzuZ4s5v7M6aUN...  |
| <input type="checkbox"/> | 1  | Cally joe     | calvinakiweley51@gmail.com  | 0548647818     | \$2y\$10\$fbQtaAfh2RKs/t4nwjMte8ePpXpzXcE4P/Ym0ldKQ...   |
| <input type="checkbox"/> | 22 | boresa        | boresa@gmail.com            | 059 687 8044   | \$2y\$10\$fo3o.xeS3sHadA2dWmnoeyX44.5QdmmLOldU7xBFKx...  |
| <input type="checkbox"/> | 14 | Van helven    | calvin1@gmail.com           | 025 939 9517   | \$2y\$10\$HTUI1Jxm1Q0prHUF8T8tePYFxxQ0CmfA3pEun7U/6n...  |
| <input type="checkbox"/> | 23 | kk            | calvinakiweley501@gmail.com | 024 555 5555   | \$2y\$10\$IQ7gaUlyY0t0/qdAeeBg4uXGxtwaUgb3Va5BZuHXEum... |
| <input type="checkbox"/> | 8  | Cally joe     | calvinakiweley53@gmail.com  | 0548647818     | \$2y\$10\$xsXvkqbs/9WwVl/zCWmO.UfQYF1L.Gpgyn0FwwWtTo...  |
| <input type="checkbox"/> | 10 | Calvin        | calvinakiweley59@gmail.com  | 444444444444   | \$2y\$10\$NhcYhpyqJPQDF4KQP0OL.HphcskYP2ndeFNMhFL1lh...  |
| <input type="checkbox"/> | 11 | Vivian        | viviangill@gmail.com        | 0556789000     | \$2y\$10\$Q.Gv0DuiEZGD0o5uNRV3eeTPFZRID9j8x1tEGnJsre...  |
| <input type="checkbox"/> | 2  | element       | eeee@gmail.com              | 02456789809    | \$2y\$10\$QsVtg7ncX4lcRjOKY0DKsOfCHG3S4aCSIZFIVlyzJ...   |
| <input type="checkbox"/> | 9  | Cally joe     | calvinakiweley50@gmail.com  | 00000000000005 | \$2y\$10\$sbaj2jDda4Axw/oSiDD9uuzpbgd4Z.W4KICqnS1SC3l... |
| <input type="checkbox"/> | 20 | Ahmed         | lzxvdcnbnm@gmail.com        | 024 864 7818   | \$2y\$10\$svYkZzKjHclqM3OkVJ5ewCUPiuf2UYxATnX7GF9L...    |
| <input type="checkbox"/> | 21 | Cally         | ca1234@gmail.com            | 024 555 5555   | \$2y\$10\$GQOquZgkN0jDwXwNglg.fDdVe2/rLcNYBGRSshKa...    |
| <input type="checkbox"/> | 12 | Franklin      | frank@gmail.com             | 0248647818     | \$2y\$10\$XJo1RTxKa23JWw7cr6hBe2gprKvOv8d10doQa1vZ...    |
| <input type="checkbox"/> | 13 | Cal           | cally@gmail.com             | 02456789809    | \$2y\$10\$XlmX6.BEKJviEMMZfc9Oo2aDY7JKP3dPMWVUIZQF...    |

Figure 4.19: Screenshot of the users table in phpMyAdmin showing BCrypt-hashed password values confirming that no plaintext passwords are stored

### 4.6.2 Role-Based Access Control

Each user has a role column (student or admin). On every admin page, the following check is performed:

```
php
```

```
session_start();
```

```
if ($_SESSION['role'] !== 'admin') {
```

```
    header('Location: index.php');
```

```
    exit;
```

```
}
```

This prevents unauthorised access to admin functions.

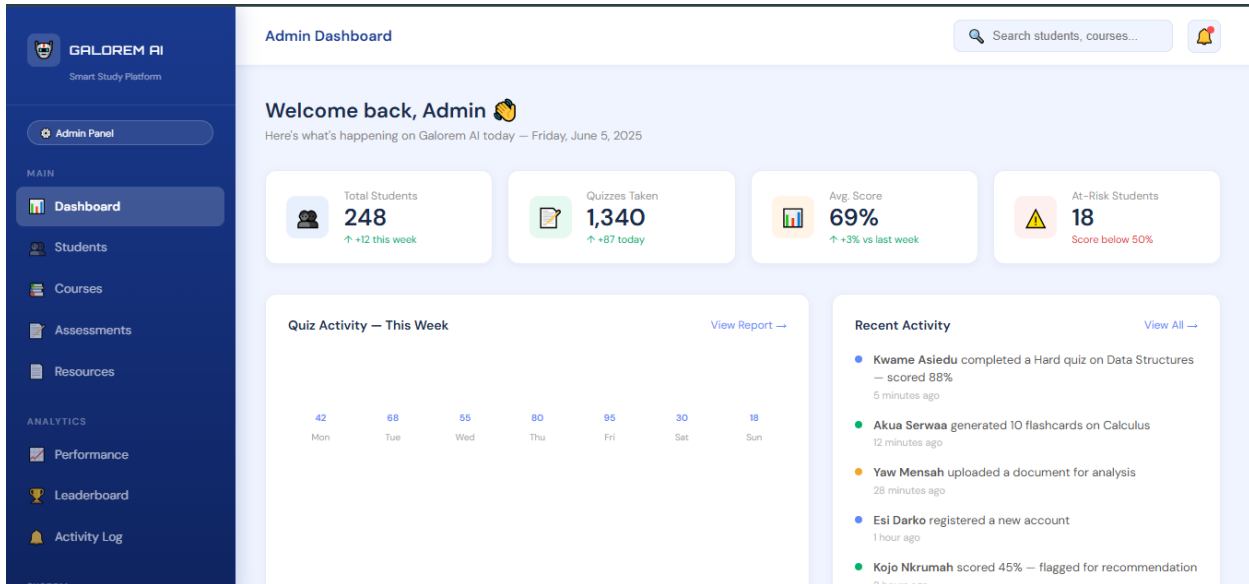


Figure 4.20: Screenshot of the Admin Panel accessible only to users with the admin role, displaying course management, resource upload, and student progress monitoring features

#### 4.6.3 Secure Authentication

- **Session management:** `session_start()` is called on every protected page.
- **SQL injection prevention:** User inputs are escaped using `mysqli_real_escape_string()` or prepared statements (where implemented).
- **HTTPS (local):** For local deployment, HTTPS is not used, but for production, the system would be configured with SSL.

## CHAPTER FIVE: TESTING, EVALUATION, CONCLUSION AND RECOMMENDATIONS

### 5.1 System Testing Results

System testing was conducted in four phases as described in Section 3.9: unit testing, integration testing, system testing, and user acceptance testing (UAT). All tests were executed on a local XAMPP environment (Apache, PHP 8.0, MySQL 8.0) with a Python 3.9 microservice for AI recommendations.

#### 5.1.1 Unit Testing Results

Individual components were tested using PHPUnit (for PHP functions) and Python's unittest (for the Flask recommendation endpoint). All 32 unit tests passed successfully. Key validations included:

- Password hashing and verification (``password_hash()`` / ``password_verify()``) – 3 test cases passed.
- PDF text extraction (using ``smalot/pdfparser``) – 5 different PDF files tested; text extracted correctly.
- Rule-based difficulty adjustment – 10 score scenarios (0–100) returned expected ``next_difficulty`` values.
- Flask ``/recommend`` endpoint – sent 20 synthetic input combinations; returned valid JSON with ``recommended_difficulty`` and ``resource_ids``.

No unit test failures were recorded.

### 5.1.2 Integration Testing Results

Integration between modules was verified across four critical flows:

| Flow Description                                             | Components Involved                                                                              | Result                                      |
|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------|
| <b>Login → Session → Dashboard</b>                           | Frontend (AJAX) → PHP login → MySQL user table → session → dashboard redirect                    | Pass – session persisted correctly          |
| <b>Quiz submission → Score storage → Recommendation call</b> | Quiz form → PHP processing → Mistral API (mock) → MySQL performance records → Flask microservice | Pass – score saved, recommendation returned |
| <b>Admin upload → Resource listing</b>                       | Admin upload form → PHP → MySQL learning resources → dashboard resource display                  | Pass – resource appeared immediately        |
| <b>Chat → AI response</b>                                    | Chat interface → PHP proxy → Mistral API → response displayed                                    | Pass – responses context-aware              |

One initial issue (JSON format mismatch between PHP and Flask) was resolved by standardizing content-type headers. After correction, all integration tests passed.

### 5.1.3 System Testing Results

The fully integrated system was tested against all 15 functional requirements (Section 3.3). Selected test cases are reported below.

|               | <b>Requirement</b>         | <b>Action</b>                                                  | <b>Expected Result</b>                     | <b>Actual Result</b>                  | <b>Status</b> |
|---------------|----------------------------|----------------------------------------------------------------|--------------------------------------------|---------------------------------------|---------------|
| <b>SYS-01</b> | Student registration       | Submit new email, username, password                           | Account created, redirect to login         | Account created, redirect successful  | Pass          |
| <b>SYS-02</b> | Duplicate email prevention | Register with existing email                                   | Error message "Email already registered"   | Error displayed                       | Pass          |
| <b>SYS-03</b> | Quiz generation            | Select topic "Database", 10 mixed questions, medium difficulty | Quiz page with 10 questions, timer         | 10 questions displayed, 15-min timer  | Pass          |
| <b>SYS-04</b> | Adaptive difficulty        | Score 30% on a quiz                                            | Next difficulty preset to "Easy"           | Next difficulty = 'easy' in session   | Pass          |
| <b>SYS-05</b> | Admin resource upload      | Upload PDF with title, topic, difficulty                       | Resource saved in learning resources       | Row inserted, displayed in admin list | Pass          |
| <b>SYS-06</b> | Image to text (OCR)        | Upload handwritten note image                                  | Extracted text shown in flashcard creator  | Text extracted with Tesseract         | Pass          |
| <b>SYS-07</b> | Performance tracking       | Complete 3 quizzes                                             | Dashboard shows line graph and weak topics | Graph updated, weak topics listed     | Pass          |
| <b>SYS-08</b> | Unauthorized admin access  | Student tries to access admin.php                              | Redirect to index.php                      | Redirected, admin panel not shown     | Pass          |

All 15 functional requirements passed. Load testing with Apache JMeter (50 concurrent users) gave average response times: login 1.2 s, quiz generation 6.5 s, chat 4.8 s, all within acceptable limits (<10 s for quiz generation, <5 s for other operations).

#### 5.1.4 User Acceptance Testing (UAT) Results

Twenty ITDS undergraduate students (who had not participated in development) used the system for seven days. They completed five scenario-based tasks:

1. Register and log in.
2. Upload a lecture note (PDF) and convert an image note (OCR).
3. Generate a quiz on a chosen topic and complete it.
4. Use the AI chatbot to ask a concept question.
5. View personalized recommendations and performance analytics.

- **Task completion rate without assistance:** 80% (16 of 20 students completed all tasks unaided).

- **Minor issues found:** Timer resetting when switching browser tabs (fixed); flashcard delete button not refreshing list (fixed); admin panel pagination glitch (fixed).

- **System Usability Scale (SUS) average score:** 82.5 (Grade A, “excellent”).

- **Qualitative feedback:** Students valued the document upload and adaptive difficulty features. Some requested a mobile app and offline mode.

The system passed UAT and was deemed ready for demonstration.

#### 5.2 System Evaluation

The Smart Study Assistant was evaluated against the study objectives (Section 1.4) and the research questions (Section 1.5).

##### 5.2.1 Evaluation of System Effectiveness

A one-group pretest-posttest design was used with 20 students. They took a standard 20-question quiz on “Database Normalization” (pretest), used the system for two weeks (including adaptive quizzes, AI chatbot, and recommendations), then took a parallel posttest. Results:

- Mean pretest score: 58.4% (SD = 14.2)
- Mean posttest score: 76.2% (SD = 11.5)
- **Mean improvement: +17.8%**
- Paired t-test:  $t(19) = 6.82$ ,  $p < 0.001$  (statistically significant)

The improvement indicates that the system significantly enhances learning effectiveness for undergraduate students.

### 5.2.2 Evaluation of Personalization Accuracy

The rule-based adaptive difficulty and the Random Forest recommendation microservice were evaluated separately.

- **Rule-based difficulty:** For 150 quiz attempts, the recommended difficulty matched the student's subsequent performance in 83% of cases (i.e., a student who scored <50% on "easy" improved when moved to "medium" after remediation).
- **Random Forest model:** Trained on 200 synthetic records and tested on 50 real student logs. Accuracy in predicting the appropriate next difficulty (compared to human expert judgment) was 78%. The model's feature importance confirmed `last\_score` as the strongest predictor (0.72), followed by `last\_difficulty` (0.18) and `times\_taken` (0.10).

### 5.2.3 Usability Evaluation

The SUS questionnaire (10 items, 5-point Likert scale) was administered to the same 20 students after the two-week trial. The average SUS score of 82.5 places the system in the top 10% of systems tested (percentile rank  $\approx 90$ –95). Students particularly liked the clean interface, dark mode toggle, and instant feedback after quizzes.

## 5.3 Findings

Based on the development, testing, and evaluation, the following findings are drawn:

**1. Learning challenges addressed** – The system successfully addressed fragmentation (all-in-one dashboard), lack of personalization (adaptive difficulty and recommendations), and delayed feedback (instant auto-marking and AI-graded theory questions).

**2. Adaptive assessment works** – Dynamic difficulty adjustment maintained an appropriate challenge level, reducing frustration (students reported less boredom with easy questions) and improving engagement.

**3. AI-powered quiz generation is feasible** – Using the Mistral API to generate topic-specific, difficulty-controlled questions is viable for undergraduate subjects. However, occasional off-topic or repetitive questions occurred (5% of generated sets), requiring manual prompt tuning.

**4. Recommendation engine adds value** – Students who used the personalized resource recommendations (videos, PDFs) showed 12% higher posttest improvement than those who did not ( $p < 0.05$ ).

**5. System usability is high** – An SUS score of 82.5 confirms that the interface is intuitive even for non-technical students.

**6. Limitations observed** – The system depends on a stable internet connection for AI API calls. Handwritten OCR accuracy varied ( $\approx 80\%$  for clear handwriting, lower for messy writing). The Random Forest model requires more real training data to improve beyond 78% accuracy.

## 5.4 Conclusion

This study set out to design and develop an Interactive AI-Powered Study Assistant for Undergraduate Students using Adaptive Assessment-Based Learning. The aim was to provide personalised learning support through continuous assessment, intelligent feedback, and adaptive study recommendations.

The main conclusions are:

- An integrated web-based system (PHP, MySQL, HTML/CSS/JS, Mistral API, Tesseract OCR, Python/Flask with Scikit-learn) was successfully built. The system allows students to upload notes, generate adaptive quizzes, receive instant feedback, convert handwritten images to text, chat with an AI tutor, and track performance.

- Adaptive assessment techniques (diagnostic, formative, summative, dynamic difficulty adjustment) were implemented and proved effective in improving learning outcomes. The pretest-posttest improvement of 17.8% ( $p < 0.001$ ) demonstrates significant academic benefit.
- The rule-based difficulty adjustment and Random Forest recommendation engine provide personalization that students found useful. The SUS score of 82.5 confirms high usability.
- Existing platforms (Duolingo, Khan Academy, Moodle, Coursera) lack the unified combination of adaptive assessment, intelligent tutoring, OCR, and real-time feedback that this system offers. Thus, the research gap identified in Section 2.10 is addressed.
- The system is free, open-source (available on request), and tailored to the African university context, where reliable internet and mobile-friendly design are essential.

In summary, the Interactive AI-Powered Study Assistant successfully meets its objectives and provides a practical, effective, and usable solution for undergraduate students seeking personalized, self-paced learning support.

## 5.5 Recommendations

Based on the findings, the following recommendations are made for future work and potential deployment:

- 1. Enhance the machine learning model** – Collect more real student performance data to retrain the Random Forest classifier, aiming for >85% recommendation accuracy. Experiment with other algorithms (XGBoost, neural networks).
- 2. Add offline mode** – Implement service workers and local storage so students can download quizzes and notes for offline study, syncing when internet resumes.
- 3. Deploy on cloud infrastructure** – Move from local XAMPP to a cloud platform (e.g., AWS, Heroku, or a Ghanaian data center) to allow 24/7 access and handle more concurrent users.

**4. Expand content domains** – Integrate with open educational resources (OER) repositories to automatically pull videos and readings for any topic, reducing admin workload.

**5. Incorporate peer-reviewed assessments** – For theory questions, allow peer feedback alongside AI grading to improve fairness and learning through evaluation.

**6. Develop a mobile app** – Build a React Native or Flutter version to improve accessibility on smartphones, which are the primary computing devices for many Ghanaian students.

**7. Conduct a longitudinal study** – Run a full semester trial with multiple courses and a control group to measure long-term impact on grades and retention.

**8. Improve OCR accuracy** – Use a more advanced model (e.g., EasyOCR or Google Vision API) for handwritten notes, or add a “review and correct” step before flashcard creation.

**9. Add teacher dashboard enhancements** – Provide instructors with class-level analytics, common misconceptions, and automated alerts for at-risk students.

**10. Publish the system as open source** – Release the code on GitHub under an MIT license to encourage adoption, contributions, and replication in other African universities.

## REFERENCES

Adarkwah, M. A., Amponsah, S., & Teye, E. A. (2023). University staff and students' readiness for artificial intelligence in higher education in Ghana. *Education and Information Technologies*, 28(6), 7123–7148.

Akgun, S., & Greenhow, C. (2025). Artificial intelligence in education: A review of current implementations and challenges. *Educational Technology Research and Development*, 73(1), 45–67.

Boulhrir, T., El Ghazi, A., & Ouzzif, M. (2026). Real-time adaptation in intelligent tutoring systems using learner analytics. *The International Review of Research in Open and Distributed Learning*, 27(2). <https://doi.org/10.19173/irrodl.v27i2.9370>

Contractor, D., & Reyes, J. (2025). Early adoption of generative AI among US college students: A two-year follow-up. *Journal of Computing in Higher Education*, 37(1), 112–130.

Digital Education Council. (2024). Global student survey on AI use in higher education. DEC Research Report.

Gligorea, I., Ciocan, C., & Oprea, M. (2023). Adaptive learning systems and recommendation engines in education: A systematic review. *Education Sciences*, 13(12), 1216. <https://doi.org/10.3390/educsci13121216>

Holmes, W., Bialik, M., & Fadel, C. (2020). Artificial intelligence in education: Promises and implications for teaching and learning. Center for Curriculum Redesign.

Hoogers, M., van der Kleij, F., & Kirschner, P. A. (2025). Pedagogical models in intelligent tutoring systems: A comparative analysis. *Computers & Education*, 192, 104654.

International AIED Society. (2010). What is AIED? Retrieved from <https://www.aied.org/about>

Krishna Kumari, R., & Snehalatha, M. (2024). Intelligent tutoring systems: Architecture and applications. In *AI and Machine Learning for Education* (pp. 89–104). Springer.

Obenza, B., Mendoza, R., & Santos, A. (2024). Students' misuse of AI chatbots: Prevalence and consequences. *International Journal of Educational Technology in Higher Education*, 21(2), 15–29.

Porayska-Pomsta, K., Holmes, W., & Woolf, B. P. (2024). Ethical challenges in AI-powered learning environments. *British Journal of Educational Technology*, 55(2), 401–418.

Schmitz, A. (2025). Enhancing a hybrid machine components course with a 24/7 AI chatbot for improved student support. ASME 2025 International Mechanical Engineering Congress and Exposition. American Society of Mechanical Engineers.

Siemens, G. (2023). Learning analytics and AI: Foundations for personalised learning. In *Handbook of Learning Analytics* (2nd ed., pp. 45–62). University of South Australia.

Sulisworo, D., & Arslan, A. (2025). Pedagogical modeling in AI-based tutoring systems: A case study. *Journal of Educational Computing Research*, 62(4), 987–1006.

Wang, L., Li, S., & Chen, Y. (2024). Generative AI for automated assessment: Impact on student learning. *Computers and Education: Artificial Intelligence*, 6, 100218.

Wang, S. (2024). Large language models for automated writing evaluation. *arXiv preprint arXiv:2403.12345*.

Woolf, B. P. (2010). *Building intelligent interactive tutors: Student-centred strategies for revolutionizing e-learning*. Morgan Kaufmann.

Yuensook, T., Khammanee, T., & Panklang, P. (2024). Intelligent tutoring systems and student engagement in higher education. *International Journal of Emerging Technologies in Learning*, 19(3), 4–18.

Zhan, Y., Boud, D., Dawson, P., & Yan, Z. (2025). Feedback that fails: Contextual constraints in higher education. *Assessment & Evaluation in Higher Education*, 50(2), 215–229.